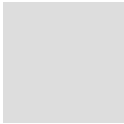


# **CSCAM**

# **HX PLC Editor 매뉴얼**



설치 / 시운전 편

---

**Serial No. : MTB-20230213**

|            |                                         |           |
|------------|-----------------------------------------|-----------|
| <b>4.1</b> | <b>HX PLC EDITOR 설치 방법 .....</b>        | <b>3</b>  |
| 4.1.1      | HX PLC EDITOR 설치 .....                  | 3         |
| <b>4.2</b> | <b>HX PLC EDITOR 사용 방법 .....</b>        | <b>3</b>  |
| 4.2.1      | HX PLC EDITOR 실행 .....                  | 3         |
| 4.2.2      | PROJECT 관리 화면 .....                     | 3         |
| 4.2.3      | LADDER 파일 열기 .....                      | 4         |
| 4.2.4      | 심벌파일 열기 .....                           | 6         |
| 4.2.5      | Project 파일의 생성 및 열기 .....               | 6         |
| 4.2.6      | Library(#lb)파일을 이용한 래더(#la) 생성 방법 ..... | 6         |
| 4.2.7      | 래더 편집 방법 .....                          | 7         |
| 4.2.8      | 편집 환경 설정 및 기능 .....                     | 11        |
| <b>4.3</b> | <b>HX PLC 사양 및 명령어 .....</b>            | <b>14</b> |
| 4.3.1      | Ladder 프로그램의 동작 순서 .....                | 14        |
| 4.3.2      | ADDRESS .....                           | 15        |
| 4.3.3      | HX PLC 명령어 .....                        | 17        |
| <b>4.4</b> | <b>HX PLC MESSAGE 파일 작성법 .....</b>      | <b>63</b> |
| 4.4.1      | HX PLC MESSAGE .....                    | 64        |
| 4.4.2      | ALARM MESSAGE (PLCAImDt.txt) .....      | 64        |
| 4.4.3      | WARNING MESSAGE (PLCOpDt.txt) .....     | 64        |

## 4.1 HX PLC EDITOR 설치 방법

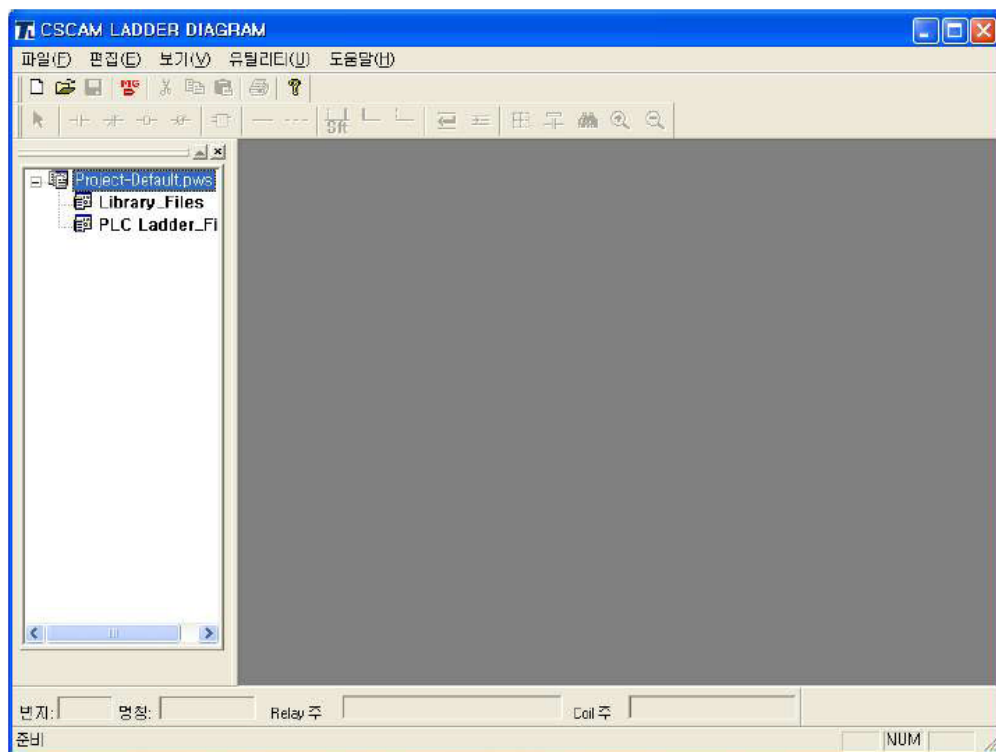
### 4.1.1 HX PLC EDITOR 설치

- (1) 800S PLC 편집기 압축 파일을 받아 임의의 폴더에 압축을 풀니다.
- (2) 하위 폴더인 Data 폴더에는 Turbohx1.ini, Turbohx1.fun, Tladder.ini 파일이 생성됩니다. 그리고, 800S 버전의 환경 파일인 Fxladder.ini, FxPlus.fun, FxPlus.ini 파일이 함께 존재합니다. 800S 버전 관련 환경 파일은 확장자가 \*.fxl 인 800S 시스템용 ladder 편집을 위해 사용됩니다.

## 4.2 HX PLC EDITOR 사용 방법

### 4.2.1 HX PLC EDITOR 실행

- (1) PLCEditor.exe 를 실행합니다.
- (2) 다음과 같은 화면이 나타납니다.



### 4.2.2 PROJECT 관리 화면

- (1) 처음 설치하는 경우는 Project 창에 파일이 존재하지 않습니다. 기본적으로 Project 에 등록된 파일정보는 PLCEditor.exe 가 있는 폴더에 Default.pws 라는 파일에 저장됩니다.

- (2) PLC Library Files

: 확장자가 #lb 인 Ladder Library 파일을 등록할 수 있습니다. Library 파일은 모듈단위로 작성한 래더 파일입니다. 예로 End1.#lb 는 단순히 'END1' 명령어만 들어있는 모듈입니다. 등록 정보엔 파일의 저장경로 정보가 함께 저장되어 있습니다.

## (3) PLC Ladder Files

: 확장자가 .#la 인 Ladder 파일을 등록합니다. Library 를 이용한 래더 파일은 유틸리티(Alt+u)의 Merge 기능을 이용하여 원하는 Library 파일들을 순서에 맞게 선택하여 새로운 래더를 만들 수 있습니다. 그 다음 Project 창에서 원하는 래더를 선택한 후 추가 작업을 하면 됩니다.

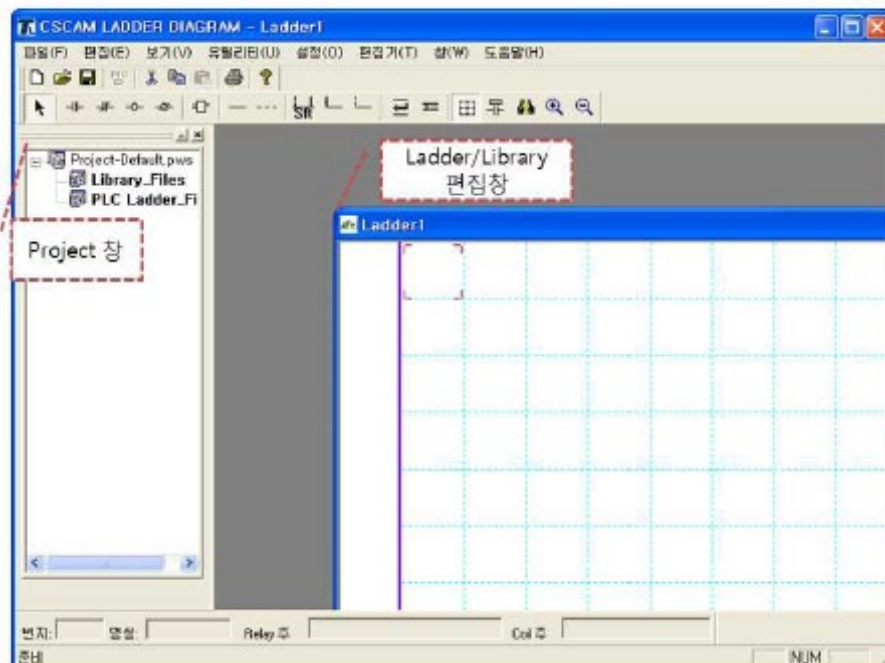
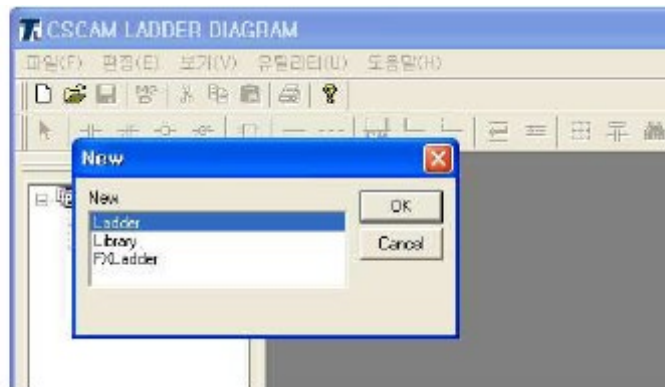
### 4.2.3 LADDER 파일 열기

#### 1) 새파일 열기

(1) 파일메뉴의 새파일 중 원하는 파일 종류를 선택합니다.(Ladder file, Library file, 800S 용 ladder file)

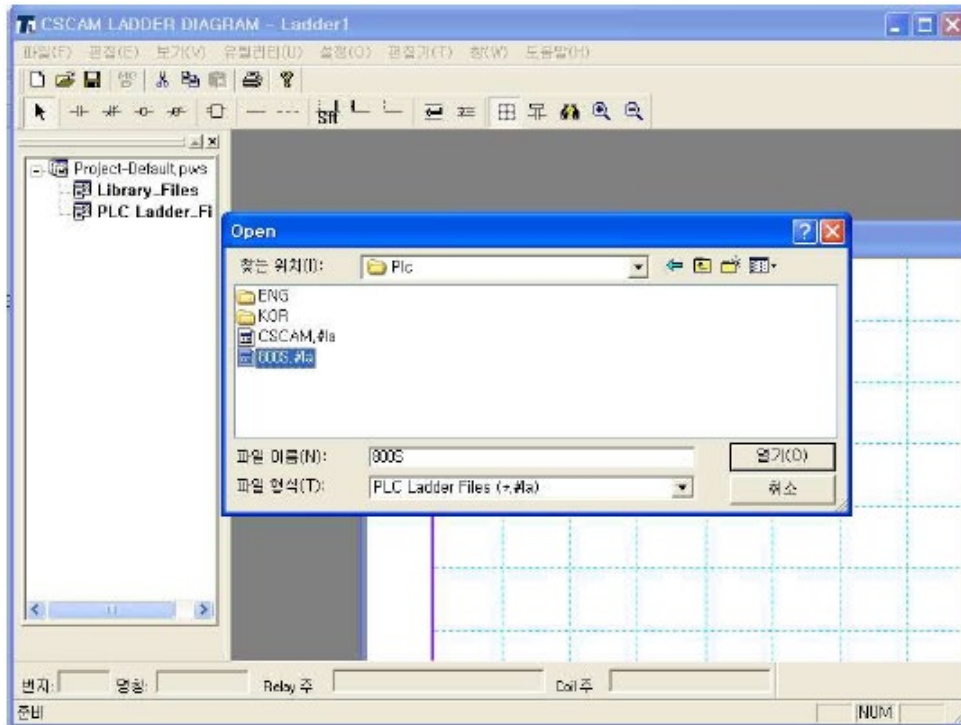
(2) Ladder1 이라는 래더창이 열리고, 툴바가 활성화 됩니다.

(3) 마우스로 툴바를 클릭하거나 F1 ~ F8 키를 이용하여 래더 다이어그램을 편집할 수 있습니다.



## 2) 기존에 작성된 래더 파일 및 라이브러리 파일 열기

- (1) 왼쪽의 Project 창의 PLC Ladder files 파일메뉴에서 원하는 래더 파일을 더블클릭하여 선택합니다. 또는 파일 메뉴에서 열기를 선택합니다. 파일선택 다이얼로그가 화면상에 나타납니다.
- (2) 확장자가 .#la 로 되어있는 파일을 선택합니다. (심벌파일(\*.#sy)은 자동으로 생성, 저장됩니다.)

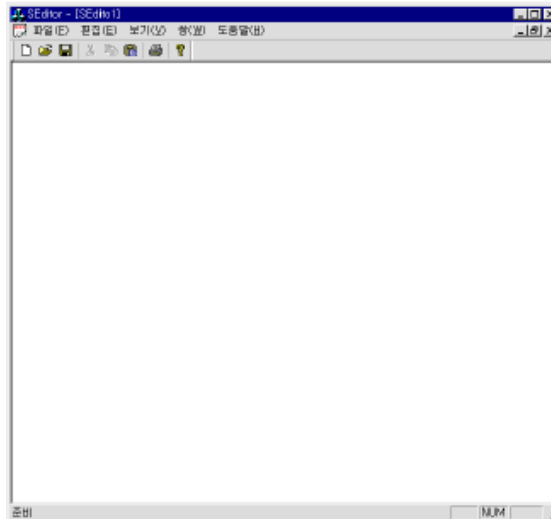


### [주의사항]

- (1) Library 파일(#lb)과 래더(#la) 파일의 이름이 같지 않도록 주의하십시오. 만약 이름이 같을 경우 Merge 를 이용하여 Ladder 파일을 생성할 때 심벌 파일의 내용이 바뀌게 됩니다.
- (2) Library 파일(#lb)과 래더(#la) 파일의 심벌파일(\*.#sy)은 자동으로 생성, 저장됩니다.

## 4.2.4 심벌파일 열기

- (1) Utility 메뉴 중 Symbol File Edit 메뉴를 선택하여 원하는 심벌 파일을 편집할 수 있습니다. (새로운 편집기 이용도 가능합니다.)
- (2) Project Workspace 에서 오른쪽 마우스에 의한 팝업 메뉴에서 Symbol File Edit 를 선택하여 원하는 심벌 파일을 편집할 수 있습니다.



## 4.2.5 Project 파일의 생성 및 열기

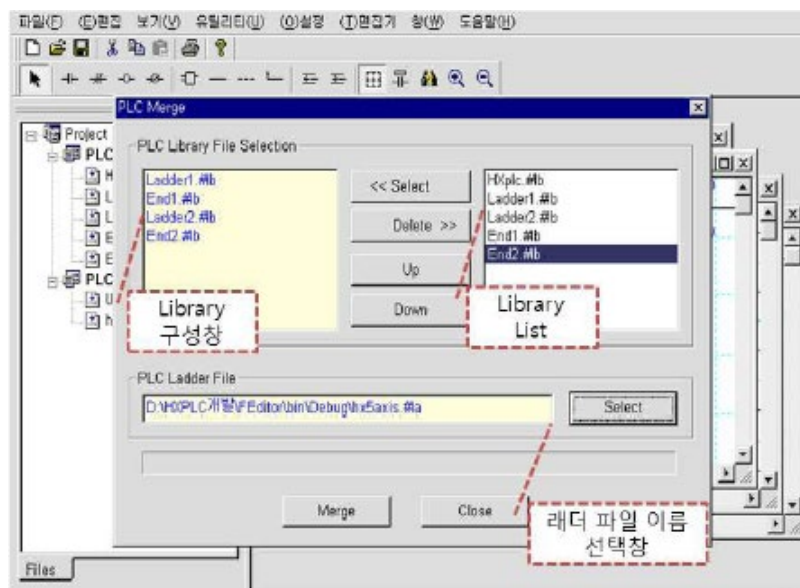
- (1) 왼쪽의 Project 창에서 오른쪽 마우스를 선택하면 프로젝트 생성 및 열기 / 저장 기능을 제공합니다. 기종별로 다양한 이름의 Ladder 파일을 Project 단위로 생성 및 관리 할 수 있습니다.
- (2) 새로운 Project 를 생성하고자 하는 경우에는 project 선택창에서 새로운 이름을 입력하면 됩니다.

## 4.2.6 Library(#lb)파일을 이용한 래더(#la) 생성 방법

: 파일 메뉴의 유틸리티의 Merge 기능을 선택합니다.

### 1) Dialog 창 이용

- (1) 파일메뉴의 Utility 에서 Merge 를 선택합니다.
- (2) 아래와 같은 대화상자가 나타납니다.



## 2) 사용 방법

- (1) 오른쪽의 Library List 창에서 원하는 모듈을 마우스를 이용하여 선택합니다. 이 때 선택한 순서대로 Ladder 파일이 구성됩니다.
- (2) “<<Select” 버튼 : List 창에서 선택된 모듈이 왼쪽의 Library 구성 창에 차례로 등록됩니다.
- (3) “Up” 버튼 : Library 구성 창에서 원하는 모듈의 순서를 맨 처음으로 합니다.
- (4) “Down” 버튼 : Library 구성 창에서 원하는 모듈의 순서를 맨 마지막으로 합니다.
- (5) “Delete” 버튼 : Library 구성 창에서 선택한 모듈을 삭제 합니다.
- (6) 아래부분의 래더파일 이름 선택창의 “Select” 버튼 : PLC Ladder File 선택창이 나타납니다. 여기에 원하는 이름을 입력하거나 Select 메뉴를 눌러 기존의 이름을 선택합니다.
- (7) “Merge” 버튼 : 래더파일이 생성되고, project 리스트에 자동 등록 됩니다.

### 4.2.7 래더 편집 방법

#### 1) 툴바 및 Function 키 사용



- (1) 원소를 입력하고자 하는 위치에 마우스를 클릭합니다.
- (2) 사각형 커서가 마우스로 클릭된 위치로 이동합니다.
- (3) 툴바에서 입력하고자 하는 원소를 클릭합니다.
- (4) 수평선과 수직선은 바로 화면에 그려지고, 나머지 원소들은 다이얼로그박스가 화면에 나타납니다. 이때 필요한 정보를 입력하고, 엔터키 또는 마우스로 OK 버튼을 클릭하면 원소가 입력됩니다.
- (5) 툴바는 마우스로 왼쪽 버튼으로 클릭합니다.

#### 1.1) RD 심벌 (F1 키)



→ 입력 심벌입니다.

#### 1.2) RDN 심벌 (F2 키)



→ 입력 반전 심벌입니다.

#### 1.3) WR 심벌 (F3 키)



→ 출력 심벌입니다.

#### 1.4) WRN 심벌 (F4 키)



→ 출력 반전 심벌입니다.

#### 1.5) Function 심벌 (F5 키)



→ 기능명령어 입력 심벌입니다. 한 RUNG 에 하나의 기능명령어만 허용됩니다.

#### 1.6) 수평선 심벌 (F6 키)



→ 수평선 심벌입니다. 수평선을 그리고 커서는 오른쪽으로 한칸 이동합니다.

## 1.7) 삭제 심벌 (F7 키)



→ 지우고자 하는 기본 원소나 기능명령어의 위치에 커서상자를 위치시킨 후 F7 키 또는 마우스로 툴바의 심벌을 클릭하면 삭제됩니다. ( 단 수직선 삭제는 Shift + F8 키에 의해서만 삭제가 가능합니다.)

## 1.8) 왼쪽수직선 우로 Shift(Shift+F7)



왼쪽수직선을 오른쪽으로 한칸 이동합니다. Shift+F7 을 누릅니다.

## 1.9) 왼쪽수직선 심벌 삽입/삭제(F8 키, Shift+F8)



→ 현재 커서 위치에서 왼쪽 방향의 수직선을 위로 그림입니다.(F8)



→ 왼쪽수직선을 삭제합니다. Shift+F8 을 누르면 지워집니다.

## 1.10) 새로운 줄 삽입/삭제 (F9 키, Shift+F9)



→ 현재 커서 위치의 아래에 새로운 빈 줄을 삽입합니다.(F9)



→ 현재 커서가 있는 빈 줄을 삭제합니다. Shift+F9 를 누르면 됩니다.

## 2) 기본 명령어, 기능 명령어 입력 방법

### 2.1) 기본명령어 입력 방법 (RD,RDN,WR,WRN 공통)

800S PLC 는 32 비트 PLC 이므로 한 어드레스당 32 비트를 표현할 수 있습니다.

“.” 다음에 오는 비트값 표기 방식은 Hexa 표기 방식을 따릅니다.

비트번호는 0 번 비트에서 31 번 비트까지 32 개의 비트값을 가집니다.

예 1) 어드레스가 X20 이고 비트가 7 번 비트 : X20.07(O), X20.7(X)

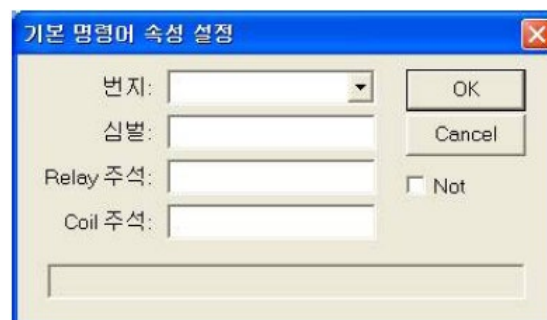
예 2) 어드레스가 X20 이고 비트가 9 번 비트 : X20.09(O), X20.9(X)

예 3) 어드레스가 X20 이고 비트가 11 번 비트 : X20.0B(O), X20.B(X)

예 4) 어드레스가 X20 이고 비트가 15 번 비트 : X20.0F(O), X20.F(X)

예 5) 어드레스가 X20 이고 비트가 31 번 비트 : X20.1F(O), X20.F(X)

순서 1. 마우스 또는 키보드를 이용해서 RD,RDN,WR,WRN 명령 중 하나를 선택하면 아래와 같이 기본명령어 속성 설정 대화상자가 나타납니다.



순서 2. 번지를 입력합니다. (반드시 입력해야 합니다.)

번지만 입력하는 경우 엔터키를 누르고, 명칭 또는 주석을 넣으려면 Tab 또는 마우스를 이용하여 커서 위치를 옮깁니다.

순서 3. 명칭(심벌)을 입력합니다. (필요한 경우에만 입력합니다, 한글도 가능합니다.)

번지에 대한 심벌의 개수는 30 자 이내로 제한을 가지나 화면에 표현될 수 있는 글자수는 6~8 자 정도이므로 가급적이면 8 자 이내로 표기를 권장합니다.

순서 4. Relay 주석을 입력합니다. (필요 시 입력합니다, 한글도 가능합니다.)

어드레스(릴레이)에 대한 주석을 30 자 이내로 입력합니다.

순서 5. Coil 주석을 입력합니다.

(한글도 가능합니다, 출력 coil 로 사용되는 경우에만 입력합니다. (X,F 는 필요 없음))

순서 6. 마우스로 OK 버튼을 누르거나 엔터키를 누릅니다.

## 2.2) 기능명령어 입력 방법



(1) 마우스 또는 키보드로 F5 를 선택 시 아래와 같이 기능명령어 속성 설정 대화상자가 나타납니다.

(2) 기능 명령어를 선택합니다.

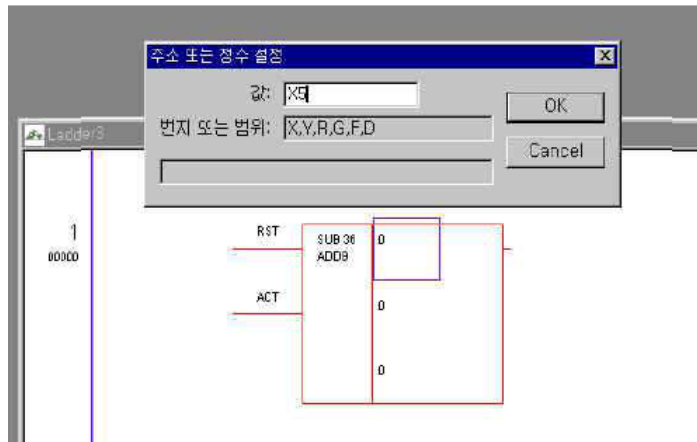
가. 키보드 사용 시 : 선택하고자 하는 기능명령어의 첫 글자를 입력하고, 아래화살표키 '↓'를 누르면 입력한 글자로 시작하는 기능명령어를 탐색합니다. 선택하고자 하는 기능명령어가 나오면 엔터를 누릅니다.

나. 마우스 사용 시 : 명령어 에디트 박스의 "t"를 마우스로 선택하면 기능명령어에 대한 알파벳 순서로 된 전체리스트가 표시됩니다. 이때 마우스를 이용하여 원하는 기능명령어를 선택하고 OK 버튼 또는 엔터키를 누릅니다.

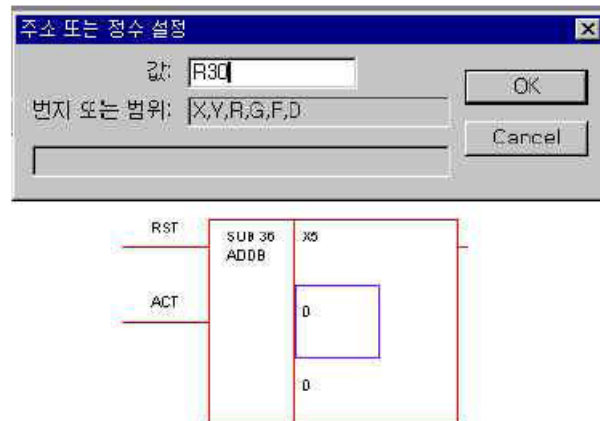
(3) 화면상에 표시된 기능명령어의 오퍼랜드를 입력합니다.

다음 그림은 SUB36 ADDB 명령어를 선택한 후 오퍼랜드를 입력하는 과정을 나타낸 것입니다. 입력할 오퍼랜드 위치에 마우스를 더블클릭 하거나 엔터키를 치면 오퍼랜드 입력 대화상자가 화면에 표시됩니다.

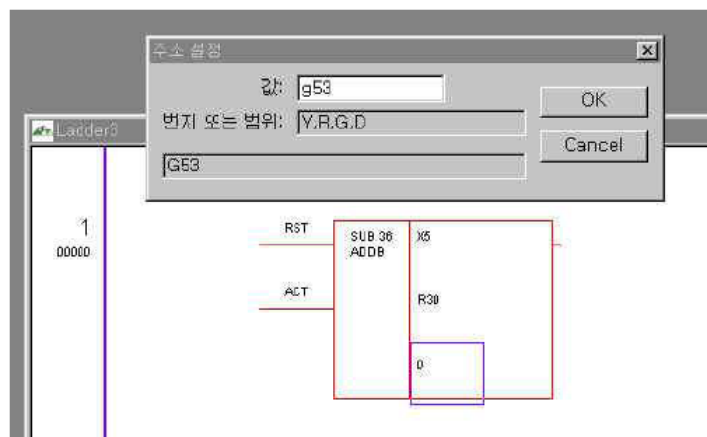
가. 첫번째 오퍼랜드 입력



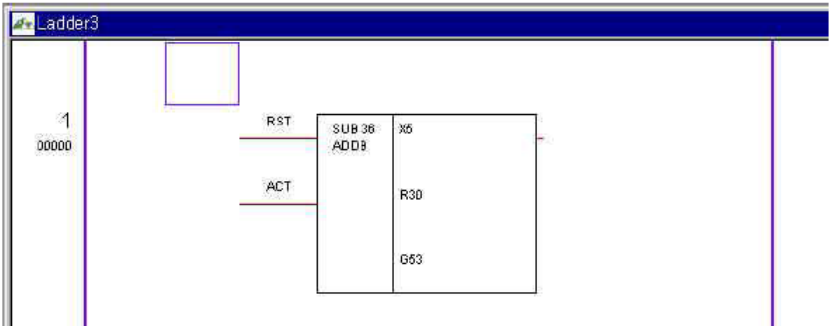
나. 두번째 오퍼랜드 입력



다. 세번째 오퍼랜드 입력



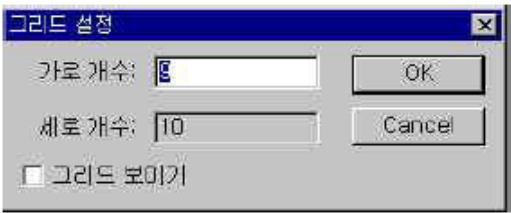
라. 오퍼랜드 입력이 완료 시 래더 화면



4.2.8 편집 환경 설정 및 기능

1) 그리드 설정  기능

역 할) 그리드 보이기 선택 및 그리드의 가로개수 설정을 합니다.  
기본값은 9 이고, 최대 20 까지 할 수 있습니다. ( 확대는 가능하나 축소는 불가능함 )

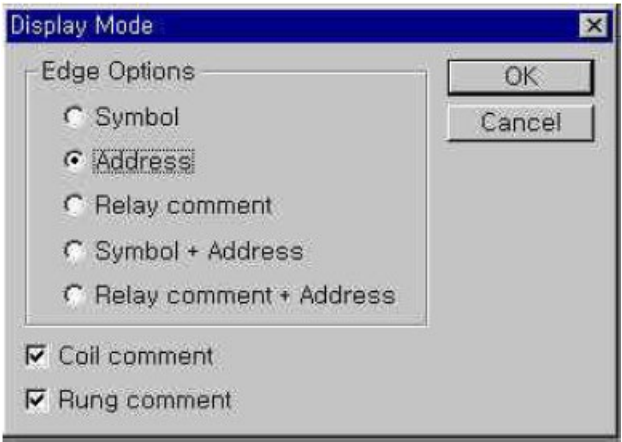


[주 의]

 편집 시 지정된 세로개수아래로 커서상자가 움직이지 않는다. 이때 세로개수를 늘려야 하는데, Line Insert 툴바 기호를 사용합니다.

2) 래더 표시 설정 기능

툴바에서  기호를 선택하면 아래와 같이 화면표시모드 설정 대화상자가 나옵니다



화면에 표시하고 싶은 주석문들을 조합해서 표시할 수 있습니다.

3) 편집 기능

## 3.1) Insert Line 기능(F9)



래더 링과 링 사이를 늘리는 역할을 합니다. 편집 시에 아주 많이 사용되는 기능입니다.

## 3.2) Delete Line 기능



래더 링과 링 사이를 줄이는 역할을 합니다

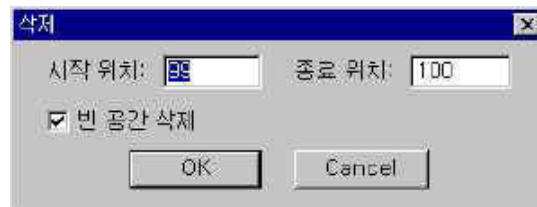
## 3.3) 현재 링 삭제 (Rung Delete : Shift + Del )

현재 커서가 위치한 링을 삭제합니다.

## 3.4) 컷 기능 (CUT : Ctrl + X )

링단위의 CUT 기능이고, 완성된 링에 대해서만 가능합니다.

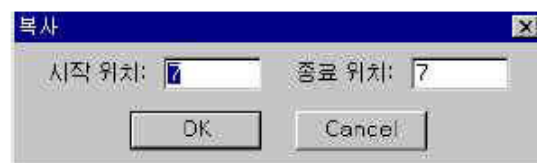
Cut 할 시작 링번호와 종료 링번호를 선택합니다.



## 3.5) 복사기능(COPY : Ctrl + C )

링단위의 COPY 기능이고, 완성된 링에 대해서만 가능합니다.

COPY 할 시작 링번호와 종료 링번호를 선택합니다.

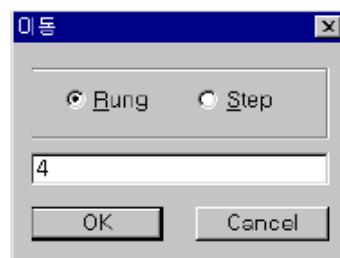


## 3.6) 붙여넣기 기능(PASTE : Ctrl + V )

링단위의 붙여넣기 기능이고, 완성된 링에 대해서만 가능합니다.

## 3.7) 행 이동(Ctrl + G)

행 이동 기능을 선택하면 아래와 같이 찾기 대화상자가 표시됩니다. 찾고자 하는 Rung 또 Step 을 입력하고 엔터키를 누르면 해당 위치로 커서가 이동합니다.



## 3.8) 찾기 기능 (Ctrl + F)

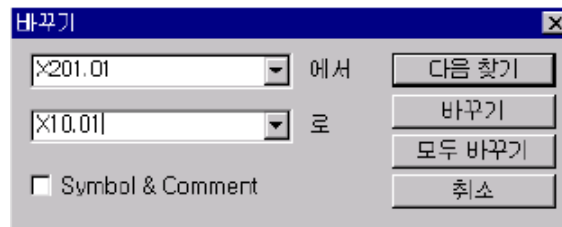


찾기 기능을 선택하면 아래와 같이 찾기 대화상자가 표시됩니다. 찾고자 하는 주소 또는 기능명령어를 입력하고 엔터키를 누르면 해당위치로 커서가 이동합니다. 계속 찾기를 하고자 하면 F11 키를 누릅니다.



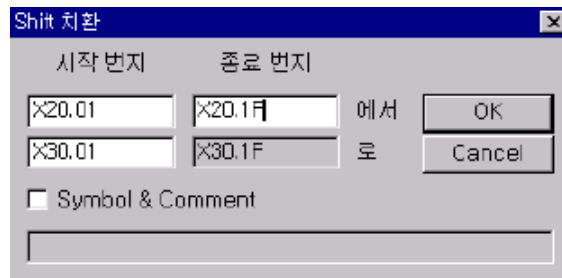
## 3.9) 바꾸기 기능 (Ctrl + E)

바꾸기 기능을 선택하면 아래와 같이 바꾸기 대화상자가 표시됩니다. 바꾸고자 하는 어드레스를 입력하고 엔터키를 누르면 바꿀 어드레스를 입력할 수 있습니다.



## 3.10) Shift 치환 기능 (Ctrl + H)

Shift 치환기능은 선택한 시작 번지부터 종료 번지까지 자동으로 다른 어드레스로 한꺼번에 변경 해주는 기능입니다. 선택하면 아래와 같이 바꾸기 대화상자가 표시됩니다. 바꾸고자 하는 시작부터 끝부분의 어드레스를 입력하고 바꿀 어드레스의 시작을 입력하면 됩니다.



## 3.11) 인쇄기능

- (1) 파일메뉴에서 인쇄를 누르면 인쇄할 항목에 대한 대화상자가 나오는데, 인쇄하고자 하는 항목을 선택합니다.
- (2) Edge List 항목은 래더 다이어그램에 사용된 심벌들에 대하여 Report 형태로 인쇄를 합니다.
- (3) Ladder 항목은 래더화면 창에 있는 그림을 그대로 인쇄합니다.
- (4) Print Dialog 보이기 항목은 인쇄 시에 프린트 다이얼로그를 보여주는지 여부를 결정합니다.
- (5) Print Dialog 보이기 항목을 선택하지 않으면 전체 인쇄를 합니다. (부분 인쇄가 불가능합니다.)

## [주의 사항]

이 항목은 Windows95,98 에서 프린트 다이얼로그에 대한 문제 발생시, 프린트 다이얼로그 기능을 막기 위해 사용됩니다.



## 3.12) 인쇄 미리 보기 기능

인쇄하기 전에 화면상으로 미리 보는 기능입니다.

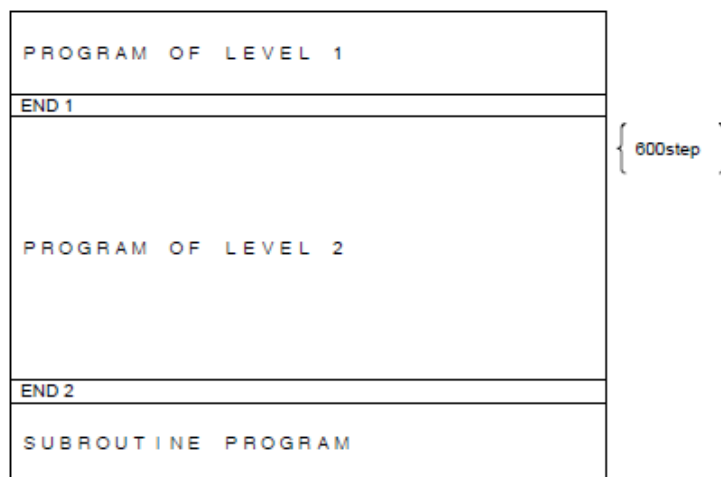
### [주의사항]

파일의 용량이 큰 경우 인쇄하는데 많은 시간이 소요될 수 있습니다.

## 4.3 HX PLC 사양 및 명령어

### 4.3.1 Ladder 프로그램의 동작 순서

#### 1) Ladder Program 의 전체 구성



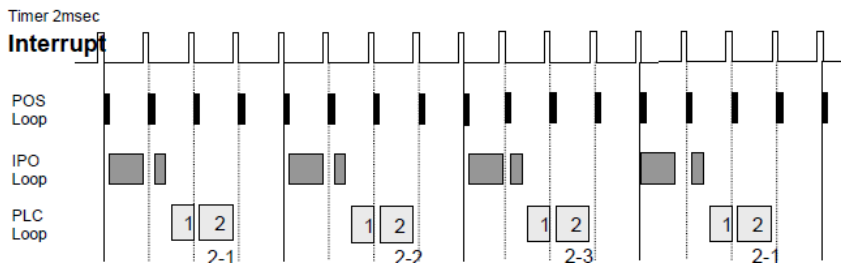
[그림 1] Ladder 프로그램 전체 구성도

- PLC Ladder Program 은 기본적으로 LEVEL-1 과 LEVEL-2 로 구성됩니다. LEVEL-2 이후에 Subroutine Program 이 올 수 있습니다.
- 프로그램의 구성에서 END1 명령과 END2 명령은 반드시 존재하여야 합니다.
- LEVEL-1 은 고속 Sequence 동작으로, 시간이 짧은 pulse 신호 등의 고속 sequence 를 처리하도록 합니다. 일정한 시간마다 처리해야 할 기능과 발생 즉시 처리해야 할 기능들을 프로그래밍하여 사용합니다. 이들 신호에는 Emergency Stop, Feed Hold, Tool Change 등과 같이 시스템에 중요한 영향을 미치는 입력을 처리하는 신호가 있습니다. LEVEL-1 은 단시간에 끝낼 수 있도록 작성하여야 합니다. 서브루틴 콜( Subroutine Call )을 할 수도 있지만, 가능하면 사용하지 않도록 합니다.
- LEVEL-2 는 LEVEL-1 이외의 Sequence 를 처리하는 루틴이다. LEVEL-2 의 처리는 600 step 씩 분할되어 처리 됩니다.
- 프로그램의 전체적인 처리 시간은 시스템에서 확인 할 수 있습니다.

2) 프로그램의 처리 시간

PLC 프로그램 처리 시간은 아래의 표와 같습니다.

예를 들어 Time 주기가 2 msec 이고, IPO 및 PLC 의 처리속도가 8 msec 일 경우 System 이 Running 상태일 때 Timing 은 다음과 같습니다.



진단 화면에서 PLC 에 대한 Scan Time 의 정보를 다음과 같이 참조 할 수 있습니다.

참고로, LEVEL-2 는 1 회 600step 씩 처리하는데 위의 Time Chart 와 같은 경우 LEVEL-2 가 3 회이기 때문에 Time Cycle 은 대략 PLC 가 1201step ~ 1800step 되는 양이라고 보면 됩니다.

- D506 은 LEVEL-2 가 모두 수행하는데 걸리는 시간 입니다.
- ('1'+2-1', '1'+2-2', '1'+2-3')
- D507 은 I/O 를 1 회 Scan Time 하는 READ 시간입니다.
- D508 은 I/O 를 1 회 Scan Time 하는 WRITE 시간입니다.
- D509 는 LEVEL-1 + LEVEL-2 (600step) 처리 시간입니다.
- D510 은 Only LEVEL-1 수행 시간입니다.
- D511 은 LEVEL-2 의 600step 처리 시간입니다.

[참 고]

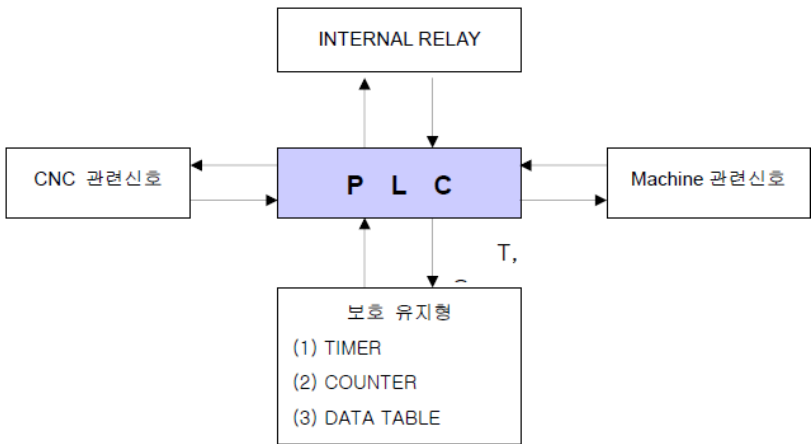
외부 입력 접점 READ 시점 :LEVEL-1 시작 시 1 회

외부 출력 접점 WRITE 시점 :LEVEL-1 끝부분 및 LEVEL-2 끝부분(상기 그림의 2-3 의 끝부분)

4.3.2 ADDRESS

1) 어드레스(ADDRESS)의 구성

어드레스(ADDRESS)는 기계측과의 입출력 신호, CNC 와의 입출력 신호, 내부 Relay, Timer, Counter, Data Table 등과 같은 신호들의 존재장소를 표시하는 번호입니다.



[그림 2] PLC 에 관련된 어드레스

X : 외부 Input  
Y : 외부 Output

## PLC Internal Data

T : Timer  
C : Counter  
R : 내부 Relay  
D : 내부 Data (Integer 형)

## PLC 와 CNC 간의 Interface Data (PLC 기준)

G : PLC 에서 CNC 로의 내부 신호 (PLC → CNC)  
F : CNC 에서 PLC 로의 내부 신호 (CNC → PLC)

## 2) 외부 입출력 신호 (입력:X, 출력 Y)

기본적으로 X 는 Read, Y 는 Read/Write 동작이 가능합니다.

X, Y Address 를 Access 할 경우 32 bit 또는 1 bit 단위로 Access 할 수 있습니다. 32 bit 단위인 경우 Address 증가는 1 이 됩니다.

800S 시스템에서는 Soft OP 를 사용할 때 내부 X Address 가 사용되는 경우도 있습니다.

| Variable Name | Unit Capacity    | Size (Byte) |  |
|---------------|------------------|-------------|--|
| X             | 256(8192 Points) | 1 Kbyte     |  |
| Y             | 256(8192 Points) | 1 Kbyte     |  |

## 3) 내부 데이터 (타이머:T, 카운터:C, 데이터:D)

T, C, D 은 모두 다 32 bit Read/Write 동작이며 기본 Address 증가는 32 bit 을 기본으로 합니다.

| Variable Name | Unit Capacity | Size (Byte) |  |
|---------------|---------------|-------------|--|
| T             | 512           | 4 Kbyte     |  |
| C             | 512           | 4 Kbyte     |  |
| D             | 4096          | 16 Kbyte    |  |

## 4) 내부 릴레이 R

R Address 를 Access 할 경우 32 bit 단위 또는 1 Bit 단위로 Access 할 수 있습니다. 32 bit 단위인 경우 Address 증가는 1 이 됩니다.

R Address 는 Size 의 절반을 기준으로 앞부분은 휘발성 영역(R0.00~1023.1F), 뒷부분은 비휘발성 영역(R1024.00~R2047.1F)으로 나뉘어지며, 휘발성 영역은 부팅 시 항상 0 으로 초기화 됩니다.

| Variable Name | Unit Capacity       | Size (Byte) |  |
|---------------|---------------------|-------------|--|
| R             | 2048 (65536 Points) | 8 Kbyte     |  |

5) 내부 시스템 입출력 신호 (입력:F, 출력:G)

F, G Address 는 PLC 와 CNC 시스템 간의 인터페이스 신호로 그 어드레스는 정의되어 있습니다.

PLC 를 기준으로 보면 F 어드레스는 CNC로부터 Read 하는 신호가 되며, G 어드레스는 CNC 로 Write 하는 신호가 됩니다.

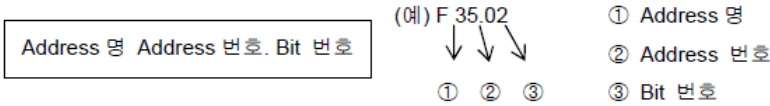
G, F Address 를 Access 할 경우 32 bit 단위 또는 1 Bit 단위로 Access 할 수 있다. 32 bit 단위인 경우 Address 증가는 1 이 됩니다.

| Variable Name |        | Unit Capacity       | Size (Byte) | Location<br>(*) System Memory<br>Size = 40 Kbyte<br>(4 X 2 X 4+ 8) |
|---------------|--------|---------------------|-------------|--------------------------------------------------------------------|
| G             | Common | 1000 (32000 Points) | 4000 byte   |                                                                    |
| F             | Common | 1000 (32000 Points) | 4000 byte   |                                                                    |

4.3.3 HX PLC 명령어

1) 접점(address)의 표현 형식

접점의 표현은 Address 명, Address 번호와 Bit 번호로 구성되며, 아래와 같은 형식으로 표현합니다. 즉, 신호에는 전부 address 가 붙습니다. 하지만 Bit 번호는 오지 않을 수도 있습니다.(T, C, D 의 경우)



2) 명령의 종류

800S PLC 의 명령에는 기본 명령과 기능 명령이 있습니다.

2.1) 기본 명령

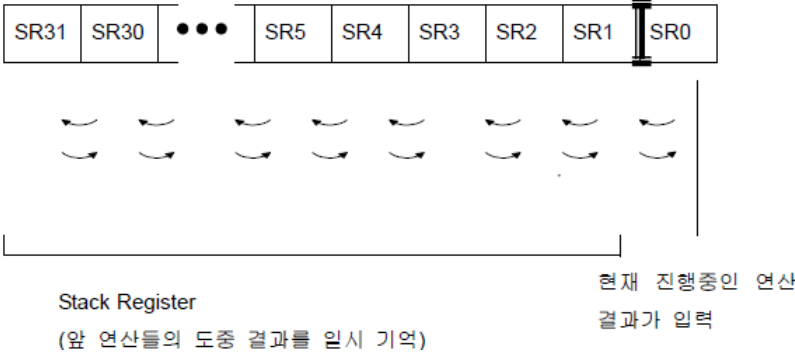
기본 명령은 sequence program 을 설계할 때, 가장 많이 사용하는 명령으로 AND, OR 등의 Bit 연산을 하는 명령으로 12 종류가 있습니다.

2.2) 기능 명령

기본 명령만으로 program 하기에 곤란한 공작 기계의 동작을 간단히 program 되도록 하는 명령이 기능 명령입니다. 기능 명령은 27 종류가 있습니다.

3) 논리 연산 결과의 기억(Stack Register)

각 연산들의 도중 결과를 일시 기억합니다. 이 Stack Register 는 아래와 같이 1Bit + 31Bit 즉, 총 32Bit 로 구성되어 있습니다.



앞 연산들의 도중 결과를 일시 기억시키는 명령(RDS 등)을 실행하면 위 그림과 같이 각 Bit 에 기억되어 있던 상태가 좌로 Shift 하여 후퇴합니다. 또 역으로 후퇴한 신호를 불러내는 명령(ANDS 등)을 실행하면 우로 Shift 가 나오게 됩니다. 가장 최후에 후퇴한 신호가 제일 먼저 나오게 됩니다.

| No. | 800S-PLC<br>명령어 | PLC<br>변환코드<br>(Hex) | 설 명                                                                  |
|-----|-----------------|----------------------|----------------------------------------------------------------------|
| 1   | RD              | 01h                  | 지정된 신호의 논리상태를 읽어 ST0 를 SET 합니다.                                      |
| 2   | RDN             | 02h                  | 지정된 신호의 논리상태를 반전하여 읽어서 ST0 를 SET 합니다.                                |
| 3   | WR              | 1Eh                  | 논리연산결과(ST0 의 상태)를 지정한 어드레스에 출력합니다                                    |
| 4   | WRN             | 1Fh                  | 논리연산결과(ST0 의 상태)를 지정한 어드레스에 반전시켜서 출력합니다.                             |
| 5   | AND             | 03h                  | 논리곱을 행합니다.                                                           |
| 6   | ANDN            | 04h                  | 지정된 신호의 논리상태를 반전하여 논리곱을 행합니다.                                        |
| 7   | OR              | 05h                  | 논리합을 행합니다.                                                           |
| 8   | ORN             | 06h                  | 지정된 신호의 논리상태를 반전하여 논리합을 행합니다.                                        |
| 9   | RDS             | 07h                  | 스택 레지스터의 내용을 1 비트 좌로 Shift 하고, 지정한 어드레스 신호의 상태를 ST0 에 SET 합니다.       |
| 10  | RDNS            | 08h                  | 스택 레지스터의 내용을 1 비트 좌로 Shift 하고, 지정한 어드레스 신호의 상태를 반전시켜서 ST0 에 SET 합니다. |
| 11  | ANDS            | 14h                  | ST0 와 ST1 의 논리곱을 ST1 에 SET 시키고, 스택레지스터의 내용을 1Bit 만큼 우로 Shift 합니다.    |
| 12  | ORS             | 15h                  | ST0 와 ST1 의 논리합을 ST1 에 SET 시키고, 스택레지스터의 내용을 1Bit 만큼 우로 Shift 합니다.    |

## 4) 기본명령의 형식

명령어 : Address 명 Address 번호 . Bit 번호

(단, ANDS 와 ORS 는 단독으로 사용됩니다.)

(주 1) [ 그림 ]과 [ 표 ]에서

①, ②, ③... 등은 A 접점을 의미하고,

①, ②, ③... 등은 B 접점을 의미합니다.

+ 는 논리합을

\* 은 논리곱을 의미합니다.

(주 2) [ 그림 ]과 [ 표 ]에서

SR0 는 스택 레지스터( Stack Register )의 0 번째 비트( Bit )를,

SR1 은 1 번째 비트,

SR2 는 2 번째 비트,

...

SR31 는 31 번째 비트를 의미합니다.

5) 기본명령어

5.1) RD ( READ )

지정한 Address 신호의 논리상태(1 또는 0)를 읽어 SR0 에 대입합니다.  
A 접점에서 Coding 을 개시할 경우에 사용합니다.



[그림 5.1] RD 의 Ladder Diagram

[표 5.1] RD 의 연산과정

| 번호 | 명령 | Address | 비고    | SR3 | SR2 | SR1 | SR0 |
|----|----|---------|-------|-----|-----|-----|-----|
| 1  | RD | X00.01  |       |     |     |     | ①   |
| 2  | WR | Y00.01  | W1 출력 |     |     |     | ①   |

5.2) RDN ( READ NOT )

지정한 Address 신호의 논리상태를 반전하여 읽어 SR0 에 대입합니다.  
B 접점에서 Coding 을 개시할 경우에 사용합니다.



[그림 5.2] RDN 의 Ladder Diagram

[표 5.2] RDN 의 연산과정

| 번호 | 명령  | Address | 비고    | SR3 | SR2 | SR1 | SR0 |
|----|-----|---------|-------|-----|-----|-----|-----|
| 1  | RDN | X1.02   |       |     |     |     | ①   |
| 2  | WR  | Y3.01   | W1 출력 |     |     |     | ①   |

## 5.3) WR ( WRITE )

논리연산결과, 즉 SR0 의 상태(1 또는 0)를 지정한 Address 에 출력합니다.

하나의 논리연산결과를 2 개 이상의 Address 에 출력할 수도 있습니다.



[그림 5.3] WR 의 Ladder Diagram

[표 5.3] WR 의 연산과정

| 번호 | 명령   | Address | 비고    | SR3 | SR2 | SR1 | SR0 |
|----|------|---------|-------|-----|-----|-----|-----|
| 1  | RD   | X1.00   |       |     |     |     | ①   |
| 2  | ANDN | Y3.02   |       |     |     |     | ①*② |
| 3  | WR   | Y2.00   | W1 출력 |     |     |     | ①*② |
| 4  | WR   | Y3.01   | W2 출력 |     |     |     | ①*② |

## 5.4) WRN ( WRITE NOT )

논리연산결과, 즉 SR0 의 상태(1 또는 0)를 반전하여 지정한 Address 에 출력합니다.



[그림 5.4] WRN 의 Ladder Diagram

[표 5.4] WRN 의 연산과정

| 번호 | 명령   | Address | 비고    | SR3 | SR2 | SR1 | SR0   |
|----|------|---------|-------|-----|-----|-----|-------|
| 1  | RD   | X1.01   |       |     |     |     | ①     |
| 2  | ANDN | X1.06   |       |     |     |     | ①*②   |
| 3  | WRN  | G3.01   | W1 출력 |     |     |     | ① * ② |

5.5) AND ( AND )

지정한 Address 신호의 논리상태와 SR0 를 논리곱을 행한 후 SR0 에 대입합니다.



[그림 5.5]AND 의 Ladder Diagram

[표 5.5] AND 의 연산과정

| 번호 | 명령  | Address | 비고    | SR3 | SR2 | SR1 | SR0    |
|----|-----|---------|-------|-----|-----|-----|--------|
| 1  | RD  | X2.02   |       |     |     |     | ①      |
| 2  | AND | X3.02   |       |     |     |     | ①*②    |
| 3  | OR  | Y1.03   |       |     |     |     | ①*②+③  |
| 4  | WR  | Y2.03   | W1 출력 |     |     |     | ① *②+③ |

5.6) ANDN ( AND NOT )

지정한 Address 신호의 논리상태를 반전하여 SR0 와 논리곱을 행한 후 대입합니다.



[그림 5.6] ANDN 의 Ladder Diagram

[표 5.6] ANDN 의 연산과정

| 번호 | 명령   | Address | 비고    | SR3 | SR2 | SR1 | SR0   |
|----|------|---------|-------|-----|-----|-----|-------|
| 1  | RDN  | X3.04   |       |     |     |     | ①     |
| 2  | AND  | X2.06   |       |     |     |     | ①*②   |
| 3  | ANDN | Y3.01   |       |     |     |     | ①*②*③ |
| 4  | WR   | G2.01   | W1 출력 |     |     |     | ①*②*③ |

5.7) OR ( OR )

지정한 Address 신호의 논리상태와 SR0 의 논리합을 행한 후 SR0 에 대입합니다.



[그림 5.7] OR 의 Ladder Diagram

[표 5.7] OR 의 연산과정

| 번호 | 명령  | Address | 비고    |
|----|-----|---------|-------|
| 1  | RDN | X0.06   |       |
| 2  | OR  | Y2.07   |       |
| 3  | WR  | G1.01   | W1 출력 |

| SR3 | SR2 | SR1 | SR0 |
|-----|-----|-----|-----|
|     |     |     | ①   |
|     |     |     | ①+② |
|     |     |     | ①+② |

## 5.8) ORN ( OR NOT )

지정한 Address 신호의 논리상태를 반전하여 SR0 와 논리합을 행한 후 SR0 에 대입합니다.



[그림 5.8] ORN 의 Ladder Diagram

[표 5.8] ORN 의 연산과정

| 번호 | 명령  | Address | 비고    |
|----|-----|---------|-------|
| 1  | RDN | X0.03   |       |
| 2  | ORN | G2.01   |       |
| 3  | WR  | G12.01  | W1 출력 |

| SR3 | SR2 | SR1 | SR0 |
|-----|-----|-----|-----|
|     |     |     | ①   |
|     |     |     | ①+② |
|     |     |     | ①+② |

## 5.9) RDS ( READ STACK )

스택 레지스터( Stack Register )의 내용을 좌로 1Bit 이동( Shift )하고, 지정한 Address 신호의 논리상태를 SR0 에 대입합니다.

지정한 신호가 A 점점일 때에 사용합니다.



[그림 5.9] RDS 의 Ladder Diagram

[표 5.9] RDS 의 연산과정

| 번호 | 명령  | Address | 비고    | SR1 | SR0     |
|----|-----|---------|-------|-----|---------|
| 1  | RD  | X2.01   |       |     | ①       |
| 2  | AND | Y1.07   |       |     | ①*②     |
| 3  | RDS | Y3.01   |       | ①*② | ③       |
| 4  | AND | X1.01   |       | ①*② | ③*④     |
| 5  | ORS |         |       |     | ①*②+③*④ |
| 6  | WR  | G2.01   | W1 출력 |     | ①*②+③*④ |

5.10) RDNS ( READ NOT STACK )

스택 레지스터( Stack Register )의 내용을 좌로 1Bit 씩 이동( Shift )하고, 지정한 Address 신호의 논리상태를 반전하여 읽어 SR0 에 대입합니다.

지정한 신호가 B 점점일 때에 사용합니다.

사용 예



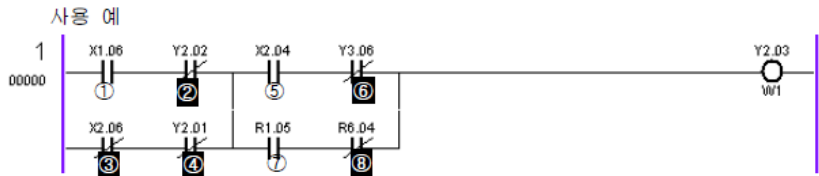
[그림 5.10] RDS 의 Ladder Diagram

[표 5.10] RDS 의 연산과정

| 번호 | 명령   | Address | 비고    | SR1                       | SR0                                            |
|----|------|---------|-------|---------------------------|------------------------------------------------|
| 1  | RD   | X1.01   |       |                           | ①                                              |
| 2  | ANDN | Y3.04   |       |                           | ①* <b>②</b>                                    |
| 3  | RDNS | X1.07   |       | ①* <b>②</b>               | <b>③</b>                                       |
| 4  | AND  | Y3.05   |       | ①* <b>②</b>               | <b>③</b> *↓                                    |
| 5  | ORS  |         |       |                           | ①* <b>②</b> + <b>③</b> *④                      |
| 6  | RDNS | R2.01   |       | ①* <b>②</b> + <b>③</b> *④ | <b>⑤</b>                                       |
| 7  | ANDN | R3.04   |       | ①* <b>②</b> + <b>③</b> *④ | <b>⑤</b> * <b>⑥</b>                            |
| 8  | ORS  |         |       |                           | ①* <b>②</b> + <b>③</b> *④+ <b>⑤</b> * <b>⑥</b> |
| 9  | WR   | Y1.03   | W1 출력 |                           | ①* <b>②</b> + <b>③</b> *④+ <b>⑤</b> * <b>⑥</b> |

5.11) ANDS ( AND STACK )

스택 레지스터( Stack Register )의 SR0 와 SR1 을 논리곱을 행하여 SR1 에 대입하고, 스택 레지스터의 내용을 1Bit 씩 우로 이동( Shift )합니다.



[그림 5.11] RDNS 의 Ladder Diagram

[표 5.11] RDNS 의 연산과정

| 번호 | 명령   | Address | SR3     | SR2     | SR0                     |
|----|------|---------|---------|---------|-------------------------|
| 1  | RD   | X1.06   |         |         | ①                       |
| 2  | ANDN | X2.02   |         |         | ①*②                     |
| 3  | RDNS | X2.06   |         | ①*②     | ③                       |
| 4  | ANDN | Y2.01   |         | ①*②     | ③*④                     |
| 5  | ORS  |         |         |         | ①*②+③*④                 |
| 6  | RDS  | X2.04   |         | ①*②+③*④ | ⑤                       |
| 7  | ANDN | Y3.06   |         | ①*②+③*④ | ⑤*⑥                     |
| 8  | RDS  | R1.05   | ①*②+③*④ | ⑤*⑥     | ⑦                       |
| 9  | ANDN | R6.04   | ①*②+③*④ | ⑤*⑥     | ⑦*⑧                     |
| 10 | ORS  |         |         | ①*②+③*④ | ⑤*⑥+⑦*⑧                 |
| 11 | ANDS |         |         |         | (①*②+③*④)<br>*(⑤*⑥+⑦*⑧) |
| 12 | WR   | Y2.03   |         |         |                         |

5.12) ORS ( OR STACK )

스택 레지스터( Stack Register )의 SR0 와 SR1 을 논리합을 행하여 SR1 에 대입하고, 스택 레지스터의 내용을 1Bit 씩 우로 이동( Shift )합니다.

사용 예

[그림 4.11]와 [표 4.11] 참조

6) PLC 기능 명령

(형식)    기능명령어코드    OP1    OP2    OP3    ,.....

주)    기능명령어마다 오퍼랜드의 개수는 차이가 있습니다.

| 명령어<br>코드 | 명령어   | 변수<br>개수 | 변수형식 | 입력<br>개수 | 출력<br>개수 | 설  명           | 비  고 |
|-----------|-------|----------|------|----------|----------|----------------|------|
| 1         | END1  | 0        |      | 0        | 0        | 제 1 레벨 프로그램 종료 |      |
| 2         | END2  | 0        |      | 0        | 0        | 제 2 레벨 프로그램 종료 |      |
| 3         |       |          |      |          |          |                |      |
| 4         |       |          |      |          |          |                |      |
| 5         |       |          |      |          |          |                |      |
| 6         |       |          |      |          |          |                |      |
| 7         |       |          |      |          |          |                |      |
| 8         | ANDMV | 3        | AAR  | 1        | 0        | 논리곱 후 데이터 전송   | 변형   |
| 9         |       |          |      |          |          |                |      |
| 10        |       |          |      |          |          |                |      |
| 11        |       |          |      |          |          |                |      |
| 12        | TMRA  | 2        | RA   | 2        | 1        | 타이머처리          |      |
| 13        |       |          |      |          |          |                |      |
| 14        | DCNV  | 2        | RR   | 4        | 1        | 데이터 변환         |      |
| 15        |       |          |      |          |          |                |      |
| 16        |       |          |      |          |          |                |      |
| 17        |       |          |      |          |          |                |      |
| 18        |       |          |      |          |          |                |      |
| 19        |       |          |      |          |          |                |      |
| 20        |       |          |      |          |          |                |      |
| 21        |       |          |      |          |          |                |      |
| 22        |       |          |      |          |          |                |      |
| 23        |       |          |      |          |          |                |      |
| 24        |       |          |      |          |          |                |      |
| 25        |       |          |      |          |          |                |      |
| 26        | ROTB  | 4        | ARRR | 5        | 1        | 바이너리 회전제어      |      |
| 27        |       |          |      |          |          |                |      |
| 28        | ORMV  | 3        | AAR  | 1        | 0        | 논리합 후 데이터 전송   |      |
| 29        |       |          |      |          |          |                |      |
| 31        |       |          |      |          |          |                |      |
| 32        |       |          |      |          |          |                |      |
| 33        |       |          |      |          |          |                |      |
| 34        | DSCHB | 4        | RRRR | 2        | 1        | 바이너리 데이터 서치    |      |
| 명령어       | 명령어   | 변수       | 변수형식 | 입력       | 출력       | 설  명           | 비  고 |

| 코드 |       | 개수 |      | 개수 | 개수 |                                        |    |
|----|-------|----|------|----|----|----------------------------------------|----|
| 35 | XMOVB | 4  | RRRR | 3  | 1  | 바이너리 인덱스수식 데이터 전송                      |    |
| 36 | ADDB  | 3  | AAR  | 2  | 1  | 바이너리 가산                                | 변형 |
| 37 | SUBB  | 3  | AAR  | 2  | 1  | 바이너리 감산                                | 변형 |
| 38 | MULB  | 3  | AAR  | 2  | 1  | 바이너리 승산                                | 변형 |
| 39 | DIVB  | 3  | AAR  | 2  | 1  | 바이너리 제산                                | 변형 |
| 40 |       |    |      |    |    |                                        |    |
| 41 | RDREG | 3  | ANR  | 1  | 0  | Read Macro Var. : Reg, cnt, Start Addr |    |
| 42 | WRREG | 2  | RN   | 1  | 0  | Write Macro Var. : Addr, Reg           |    |
| 43 |       |    |      |    |    |                                        |    |
| 44 |       |    |      |    |    |                                        |    |
| 47 | MOVB  | 2  | AR   | 1  | 0  | 바이너리 데이터 전송                            |    |
| 48 |       |    |      |    |    |                                        |    |
| 49 |       |    |      |    |    |                                        |    |
| 50 |       |    |      |    |    |                                        |    |
| 51 |       |    |      |    |    |                                        |    |
| 52 |       |    |      |    |    |                                        |    |
| 53 |       |    |      |    |    |                                        |    |
| 54 |       |    |      |    |    |                                        |    |
| 55 | CTRA  | 2  | RA   | 4  | 1  | 카운터 처리                                 | 변형 |
| 56 | COINA | 2  | AA   | 1  | 1  | 일치판정                                   |    |
| 57 |       |    |      |    |    |                                        |    |
| 58 |       |    |      |    |    |                                        |    |
| 59 |       |    |      |    |    |                                        |    |
| 60 |       |    |      |    |    |                                        |    |
| 61 |       |    |      |    |    |                                        |    |
| 62 |       |    |      |    |    |                                        |    |
| 63 |       |    |      |    |    |                                        |    |
| 64 |       |    |      |    |    |                                        |    |
| 65 | CALL  | 1  | R    | 1  | 0  | 서브루틴 콜                                 |    |
| 66 |       |    |      |    |    |                                        |    |
| 67 |       |    |      |    |    |                                        |    |
| 68 | JMPB  | 1  | R    | 1  | 0  | JUMP                                   |    |
| 69 | LBL   | 1  | R    | 0  | 0  | LABEL                                  |    |
| 70 |       |    |      |    |    |                                        |    |
| 71 | SP    | 1  | R    | 0  | 0  | 서브루틴시작                                 |    |
| 72 | SPE   | 0  |      | 0  | 0  | 서브루틴종료                                 |    |

| 명령어 코드 | 명령어 | 변수 개수 | 변수형식 | 입력 개수 | 출력 개수 | 설 명          | 비 고 |
|--------|-----|-------|------|-------|-------|--------------|-----|
| 73     |     |       |      |       |       |              |     |
| 74     | INV | 1     | R    | 1     | 1     | 비트반전         |     |
| 75     | EQU | 2     | AA   | 1     | 1     | EQUAL        |     |
| 76     | GT  | 2     | AA   | 1     | 1     | GREATER THEN |     |
| 77     | LT  | 2     | AA   | 1     | 1     | LESS THEN    |     |
| 78     | SFR | 2     | AA   | 1     | 1     | SHIFT RIGHT  |     |
| 79     | SFL | 2     | AA   | 1     | 1     | SHIFT LEFT   |     |

주) 변수(오퍼랜드)형식 설명  
A: 정수 또는 어드레스  
N: 정수  
R: 어드레스 (X,Y,G,F,R,T,C,D,A,K)  
D: 형식오퍼랜드에 의해 정수타입인지 어드레스 타입인지 결정됩니다.  
E: 형식오퍼랜드 0: 정수, 1:어드레스  
B: 형식오퍼랜드 1: BYTE, 2:WORD(2BYTE), 4: DOUBLE WORD(4BYTE)  
C: 형식오퍼랜드 A 와 B 가 복합된 형태  
(10 진수 4 자리중 4 번째(가장왼쪽) DIGIT 은 A 형식, 1 번째(오른쪽) DIGIT 은 B 형식)

예) 0004: 정수이면서 4BYTE 연산  
어드레스이면서 2BYTE 연산

6.1) END1 ( SUB 1: LEVEL-1 SEQUENCE PROGRAM 의 종료 )

[ 명령어: SUB 1 END1 ]

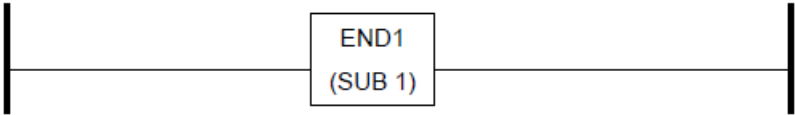
(기능)

이 명령은 SEQUENCE PROGRAM 에서 반드시 사용하는 명령이며 LEVEL-1 SEQUENCE 의 종료에 지정합니다. 만약 LEVEL-1 SEQUENCE 가 없는 경우는 LEVEL-2 SEQUENCE 선두에 반드시 위치하여야 합니다.

(형식)

END1

(리더)



6.2) END2 ( SUB 2: LEVEL-2 SEQUENCE PROGRAM 의 종료 )

[ 명령어: SUB 2 END2 ]

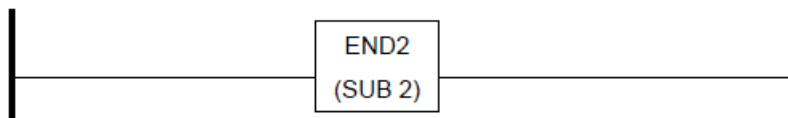
(기능)

이 명령은 SEQUENCE PROGRAM 에서 반드시 사용하는 명령이며 LEVEL-2 SEQUENCE 의 종료에 위치합니다. 프로그램 수행에 반드시 존재해야 합니다.

(형식)

END2

(래더)



## 6.3) ANDMV ( SUB 8: 논리곱 후 전송 )

[ 명령어: SUB 8 ANDMV ]

(기능)

일반적으로 입력 신호 어드레스(X Address)에 여러 종류의 데이터가 혼재되어 있는 경우 필요한 데이터 부분의 비트를 MASKING 하여 논리곱을 행한 후 지정된 어드레스로 출력하는 명령입니다.

(형식)

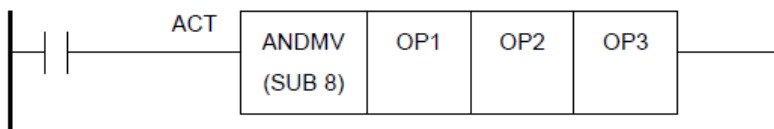
| ANDMV | OP1                             | OP2 | OP3 |
|-------|---------------------------------|-----|-----|
| OP1   | 입력 데이터 오퍼랜드                     |     |     |
| OP2   | 논리곱 데이터 오퍼랜드                    |     |     |
| OP3   | OP1 과 OP2 를 논리곱 한 결과를 저장하는 어드레스 |     |     |

| ANDMV | OP1  | OP2  | OP3  |
|-------|------|------|------|
|       | 어드레스 | 어드레스 | 어드레스 |
|       | 어드레스 | 상수   | 어드레스 |
|       | 상수   | 어드레스 | 어드레스 |
|       | 상수   | 상수   | 어드레스 |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|----------------------------|---|---|---|---|---|---|---|-----|
|      | X                          | Y | R | G | F | D | T | C | INT |
| OP1  | O                          | O | O | O | O | O | O | O | O   |
| OP2  | O                          | O | O | O | O | O | O | O | O   |
| OP3  | X                          | O | O | O | X | O | O | O | X   |

(래더)



(설명)

제어조건

실행명령 : ACT

ACT = 0 : ANDMV 명령을 실행하지 않습니다.

ACT = 1 : ANDMV 명령을 실행합니다.

제 1 오퍼랜드 (OP1): 입력 데이터 오퍼랜드

상수값의 범위 : -2147483648 ~ 2147483647

제 2 오퍼랜드 (OP2): 논리곱 데이터 오퍼랜드

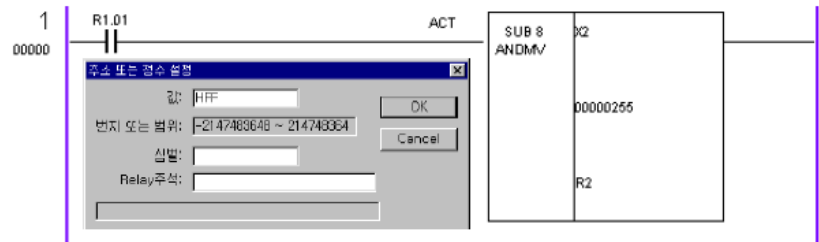
상수값의 범위 : -2147483648 ~ 2147483647

제 3 오퍼랜드(OP3): 출력 어드레스

ACT 가 1 인 경우에만 수행합니다.

제 1 오퍼랜드와 제 2 오퍼랜드와 논리곱 한 결과를 저장합니다.

(ANDMV 의 사용 예)



- 상위 래더 다이어그램은 입력 접점 중 어드레스 X2 의 32 비트 데이터 중 하위 8 비트의 데이터만을 논리곱하여 어드레스 R2 로 출력하기 위해 작성한 부분입니다.
- 논리곱 데이터 오퍼랜드 OP2 는 X,Y,R,G,F,D,T,C,정수를 사용할 수 있으며, 예에서는 정수를 사용하였습니다. PLC EDITOR 에서 논리곱 데이터 입력 시 ‘H’를 사용하면 16 진수로 입력이 가능합니다.
- (‘H’를 사용하지 않은 경우는 정수 데이터가 됩니다.)
- 위 예에서는 하위 8 비트를 논리곱 하기 위해 ‘HFF’를 입력하였으나 만일 16, 17 번째 비트에 해당하는 부분만을 논리곱하기 위해서는 H30000 를 입력하면 됩니다.

6.4) TMRA ( SUB 12 : 타이머 처리 )

[ 명령어: SUB 12 TMRA ]

(기능)

ON Delay 타이머와 PRESET 타이머 기능을 행합니다.

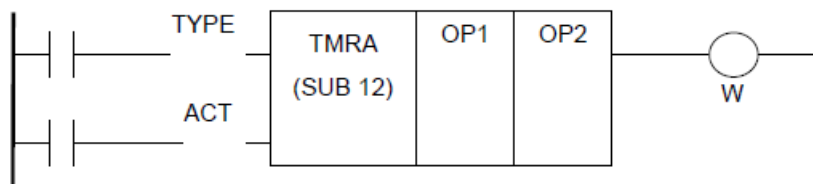
(형식)

| TMRA |                                   | OP1 | OP2 |
|------|-----------------------------------|-----|-----|
| OP1  | 타이머 번호 지정 어드레스 ( 내용은 타이머 경과치입니다.) |     |     |
| OP2  | 타이머 설정치 레지스터                      |     |     |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|----------------------------|---|---|---|---|---|---|---|-----|
|      | X                          | Y | R | G | F | D | T | C | INT |
| OP1  | X                          | X | X | X | X | X | O | X | X   |
| OP2  | X                          | X | X | X | X | O | X | X | O   |

(래더)



(설명)

## 1) 제어조건

a) 타이머 종류 지정 : TYPE

TYPE = 0 : OFF Delay Timer

TYPE = 1 : ON Delay Timer

b) 실행지령 : ACT

ACT = 0 : 디코드 결과 출력을 OFF 합니다.

ACT = 1 : 디코드 처리를 행합니다.

제 1 오퍼랜드 (OP1): 타이머 번호 지정 어드레스

c) T 어드레스만 사용가능

d) 용도 : t 어드레스 번호를 지정합니다. 지정된 T 어드레스 번호의 내용은 타이머 동작 시 경과값이 됩니다.

제 2 오퍼랜드 (OP2): 타이머 설정치 어드레스

e) 상수값의 범위 : 0 ~  $2^{31}-1$  (1 ~ 2147483647)

f) D, R 어드레스와 정수만 사용 가능합니다.

g) 어드레스 지정 시는 가변 타이머 용으로 사용 가능하고, 정수 사용 시는 고정 타이머가 됩니다.

h) Timer 의 설정 시간은 1/1000 sec 단위이며, 약 8 msec 단위로 처리 됩니다.

결과 출력 : W

i) OFF Delay Timer 인 경우는 ACT=1 이면 출력 W=1 로 하고, 경과치가 설정치에 도달하면 출력 W=0 으로 합니다.

j) ON Delay Timer 인 경우는 ACT=1 이면 출력 W=0 로 하고, 경과치가 설정치에 도달하면 출력 W=1 으로 합니다.

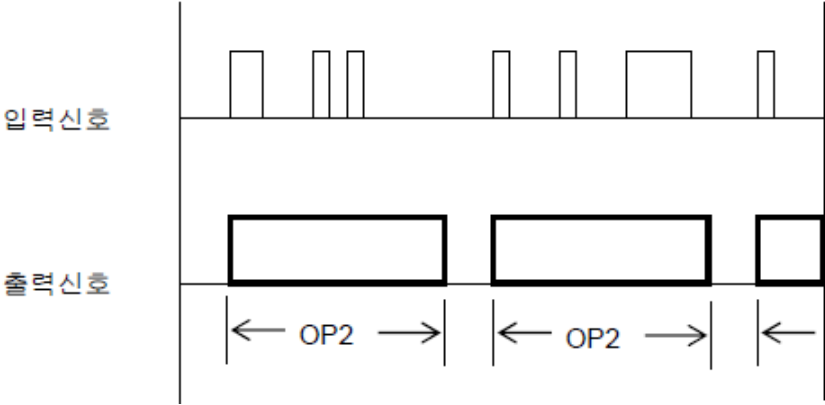
[표 4.11] TIMER 의 연산과정

| 번호 | 명령  | Address | 비고    | SR1      | SR0      |
|----|-----|---------|-------|----------|----------|
| 1  | RD  | TYPE    |       |          | TIMER 종류 |
| 2  | RDS | ACT     |       | TIMER 종류 | 입력신호     |
| 3  | TMR | OP1 OP2 | TIMER | TIMER 종류 | 입력신호     |
| 4  | WR  | W       | W 출력  | TIMER 종류 | W        |

TIMER 종류(TYPE) = 0 (OFF Delay TIMER)

- 입력신호(SR0) = ON(1) : TIMER 가 작동합니다.
- 입력신호(SR0) = OFF(0): TIMER 는 작동하지 않으며 OFF 를 출력합니다.

입력신호가 OFF(0)인 상태에서 한번 ON(1)이 되면, 출력을 ON 시킵니다. 그리고 현재치가 증가하여 설정치가 되면 출력을 OFF 시키고 TIMER 를 Reset 시킵니다.

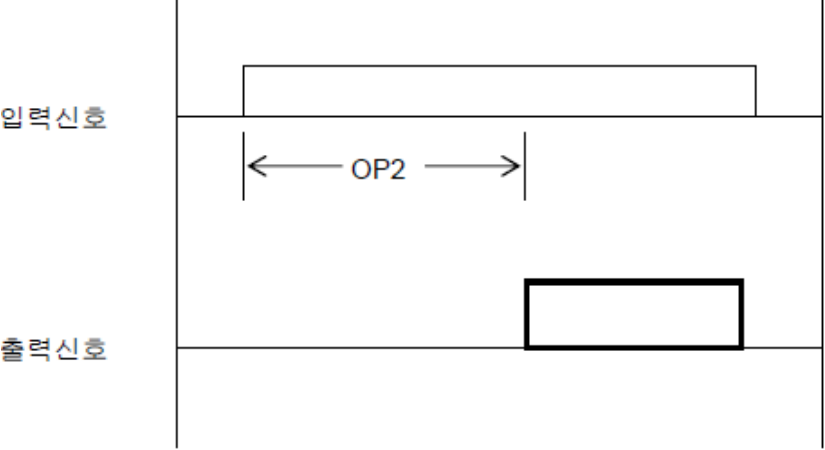


[그림 5.19] OFF TIMER 의 출력

TIMER 종류(TYPE) = 1 (ON DELAY TIMER)

- 입력신호(SR0) = ON(1) : TIMER 가 작동합니다.
- 입력신호(SR0) = OFF(0): TIMER 는 작동하지 않으며 OFF 를 출력합니다.

입력신호가 ON 인 동안 현재치를 증가하여 TIMER 의 설정치에 도달하면, 입력신호가 ON 인 동안만 출력(Stack Register 의 SR0)을 ON 으로 대입하며 도달하기 전까지는 OFF 를 출력합니다. 그리고 입력이 OFF 되면 출력을 OFF 시키고 다시 TIMER 를 Reset 시킵니다. 또한 현재치도 0 으로 대입됩니다.



[그림 5.18] ON DELAY TIMER 의 출력

현재치가 설정치에 도달하기 전에 입력신호가 OFF 되면 TIMER 를 Reset 시킵니다. 그리고 현재치도 0 으로 대입됩니다.

입력신호에 의하여 출력이 ON 되어 있는 상태에서의 입력신호는 무시됩니다.

OP1 의 Address 번호는 항상 [T<timer 번호>]가 지령 되어 하며, OP2 는 TIMER 의 설정치입니다. 이 설정치는 1/1000 초 단위로 입력합니다. 따라서 두 번째 OPERAND 가 상수인 경우에 설정치의 범위는  $0 \sim 2^{31}-1$  이므로  $0 \sim [(2^{31}-1)/1000]$ 초까지 가능합니다.

## 6.5) DCNV ( SUB 14: DATA 변환)

[ 명령어: SUB 14 DCNV ]

(기능)

BINARY CODE 를 BCD CODE 로 변환하는 명령입니다.

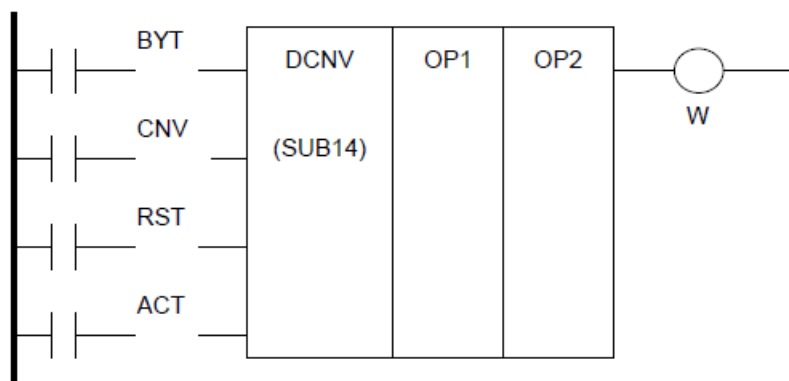
(형식)

|     | DCNV         | OP1 | OP2 |
|-----|--------------|-----|-----|
| OP1 | 입력데이터 어드레스   |     |     |
| OP2 | 변환결과 출력 어드레스 |     |     |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   |     |
|------|----------------------------|---|---|---|---|---|---|---|-----|
|      | X                          | Y | R | G | F | D | T | C | INT |
| OP1  | O                          | O | O | O | O | O | X | X | O   |
| OP2  | X                          | O | O | O | X | O | X | X | X   |

(래더)



(설명)

### 1) 제어조건

#### a) DATA 크기지정 : BYT

BYT = 0 : 처리 DATA 가 2 BYTE (16 Bit)

BYT = 1 : 처리 DATA 가 4 BYTE (32 Bit)

#### b) 변환형식 지정 : CNV

CNV = 0 : BINARY CODE 를 BCD CODE 로 변환합니다.

CNV = 1 : BCD CODE 를 BINARY CODE 로 변환합니다.

#### c) 리세트 : RST

RST = 0 : 리세트 해제합니다.

RST = 1 : 에러 출력 W 를 리세트합니다.

d) 실행지령 :ACT

ACT=0 : 데이터 변환을 행하지 않습니다.

ACT=1 : 데이터 변환을 행합니다.

2) 결과 출력 :W

W=0 : 정상

W=1 : 변환 에러

입력 데이터에 BCD 데이터가 있어야 하는데 바이너리 데이터가 있는 경우 또는 바이너리 데이터를 BCD 데이터로 변화 시 미리 지정한 DATA의 길이를 넘는 경우 W=1로 됩니다.

6.6) ROTB ( SUB 26 : 회전제어 처리 )

[명령어: SUB 26 ROTB]

(기능)

공구대, ATC, 회전테이블 등의 회전체 제어에 사용하는 명령으로, 근접 회전 방향의 판별, 현재 위치와 목표위치 사이의 step 수 산출, 목표치와 한 step 전의 위치 산출 또는 한 step 전까지의 step 수를 산출합니다.

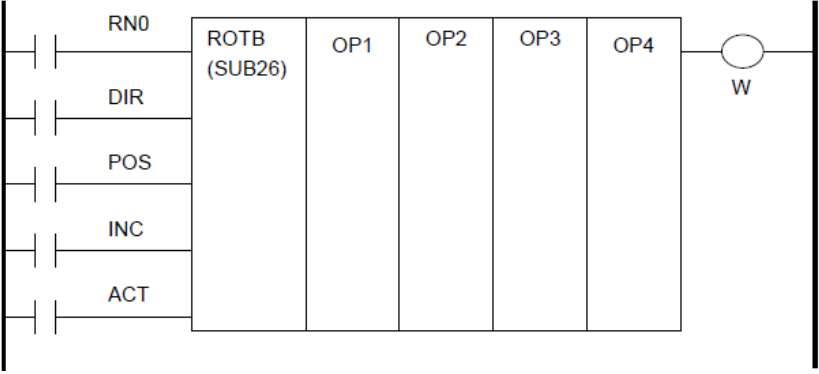
(형식)

| ROTB OP1 OP2 OP3 OP4 |              |  |  |  |
|----------------------|--------------|--|--|--|
| OP1                  | 회전체 분할 수     |  |  |  |
| OP2                  | 현재위치 어드레스    |  |  |  |
| OP3                  | 목표위치 어드레스    |  |  |  |
| OP4                  | 연산결과 출력 어드레스 |  |  |  |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|----------------------------|---|---|---|---|---|---|---|-----|
|      | X                          | Y | R | G | F | D | T | C | INT |
| OP1  | O                          | O | O | O | O | O | X | X | O   |
| OP2  | O                          | O | O | O | O | O | X | X | X   |
| OP3  | O                          | O | O | O | O | O | X | X | X   |
| OP4  | X                          | O | O | O | X | O | X | X | X   |

(래더)



(설명)

## 1) 제어조건

### a) 회전체의 시작 번호 지정 : RN0

RN0 = 0 : 회전체위치의 번호가 0 부터 시작하는 연속된 번호

RN0 = 1 : 회전체위치의 번호가 1 부터 시작하는 연속된 번호

### b) 근접 회전 방향 판별 : DIR

DIR = 0 : 근접회전방향을 판별하지 않는다. 회전방향은 FORWARD 방향입니다.

DIR = 1 : 근접회전방향을 판별한다. 회전 방향의 정의는 (6)을 참조하십시오.

### c) 연산조건의 지정 : POS

POS = 0 : 목표위치를 산출합니다.

POS = 1 : 목표위치의 1STEP 전의 위치를 산출합니다.

### d) 위치 또는 스텝수의 지정 : INC

INC = 0 : 위치의 번호를 산출합니다.

(목표위치 1 step 전의 위치 번호를 산출하는 경우에는 INC = 0 이고 POS=1 을 지정)

INC = 1 : STEP 수를 산출합니다.

(현재 위치와 목표위치의 차를 산출하는 경우에는 INC = 1 로 하고 POS=0 을 지정)

### e) 실행명령 : ACT

ACT = 0 : ROT 명령을 실행하지 않습니다. 출력 W 는 변화하지 않습니다.

ACT = 1 : ROT 명령을 실행합니다.

통상은 ACT=0 으로 하고 연산 결과가 필요할 때에 ACT=1 로 합니다.

제 1 오퍼랜드(OP1) : 회전체 분할 수

회전체의 분할 수를 지정합니다.

상수값의 범위 : 1 ~ 9999

제 2 오퍼랜드(OP2) : 현재위치 어드레스

현재 위치가 기억되어 있는 어드레스를 지정합니다.

제 3 오퍼랜드(OP3) : 목표위치 어드레스

목표위치가 기억되고 있는 어드레스를 지정합니다.

예로 CNC 의 출력으로 T code 가 들어있는 address 가 될 수 있습니다.

제 4 오퍼랜드(OP4) : 연산결과 출력 어드레스

회전해야 하는 스텝 수, 1 스텝 앞의 스텝 수 또는 목표위치의 1 스텝 앞의 위치를 산출합니다.

결과 출력 : W

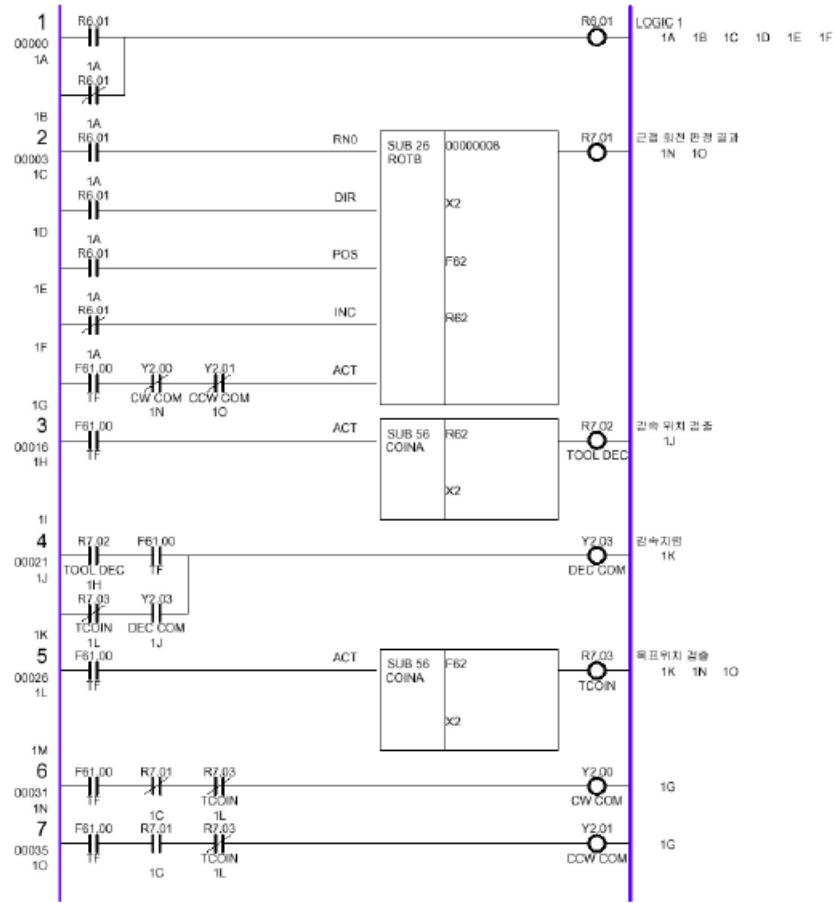
근접회전을 하는 경우의 회전 방향이 W 로 출력됩니다.

W = 0 : 회전방향이 FORWARD (회전체의 index 가 증가하는 방향)

W = 1 : 회전방향이 REVERSE (회전체의 index 가 감소하는 방향)

(사용 예)

8 개의 위치 회전체에서 근접회전을 해야 하고, 1STEP 전 위치에서 감속해야 하는 경우에 대한 래더 다이어그램의 예입니다.



RN0, DIR = 1 이므로 회전체위치의 번호가 1 부터 시작하는 연속된 번호로 근접회전방향을 판별하며, POS=1,INC=0 이므로 목표위치 1 STEP 전의 위치 번호를 산출합니다.

T 코드 개시 신호(TF) F61.0와 회전체에 정,역회전 지령이 없는 경우 ROTB 는 동작합니다.

목표위치는 OP3 어드레스 F62 로 지령되며, 회전체의 현재위치는 어드레스 X2 로 입력되며 1 STEP 전의 위치 연산 결과는 어드레스 R62 로 출력 됩니다.

감속위치의 검출과 지정 위치 판정을 위해 COINA 명령을 사용합니다.

## 6.7) ORMV ( SUB 28: 논리합 후 전송 )

[명령어: SUB 28 ORMV]

(기능)

입력 데이터에 논리합을 행한 후 지정된 어드레스로 출력하는 명령입니다.

(형식)

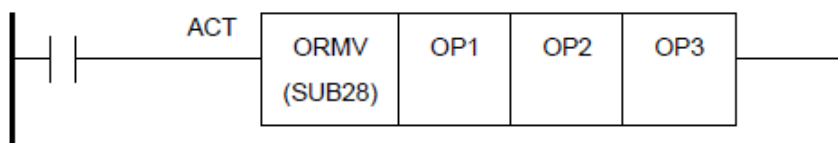
| ORMV | OP1                             | OP2 | OP3 |
|------|---------------------------------|-----|-----|
| OP1  | 입력 데이터 오퍼랜드                     |     |     |
| OP2  | 논리합 대상 오퍼랜드                     |     |     |
| OP3  | OP1 과 OP2 를 논리합 한 결과를 저장하는 어드레스 |     |     |

| 명령어  | OP1  | OP2  | OP3  |
|------|------|------|------|
| ORMV | 어드레스 | 어드레스 | 어드레스 |
|      | 어드레스 | 상수   | 어드레스 |
|      | 상수   | 어드레스 | 어드레스 |
|      | 상수   | 상수   | 어드레스 |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|----------------------------|---|---|---|---|---|---|---|-----|
|      | X                          | Y | R | G | F | D | T | C | INT |
| OP1  | O                          | O | O | O | O | O | O | O | O   |
| OP2  | O                          | O | O | O | O | O | O | O | O   |
| OP3  | X                          | O | O | O | X | O | O | O | X   |

(래더)



설명)

### 1) 제어조건

a) 실행명령 : ACT

ACT = 0 : ANDMV 명령을 실행하지 않습니다.

ACT = 1 : ANDMV 명령을 실행합니다.

### 2) 제 1 오퍼랜드(OP1) : 입력 데이터 오퍼랜드

상수값의 범위 : -2147483648 ~ 2147483647

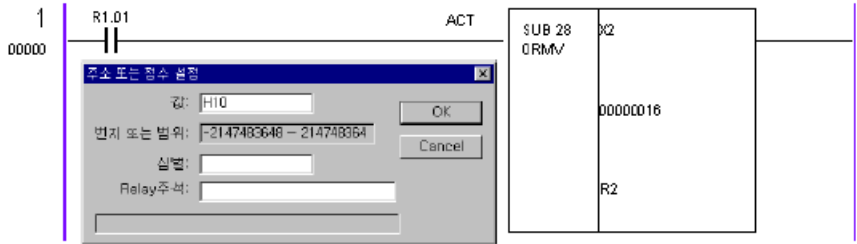
### 제 2 오퍼랜드(OP2) : 논리합 데이터 오퍼랜드

상수값의 범위 : -2147483648 ~ 2147483647

### 제 3 오퍼랜드(OP3) : 출력 어드레스

제 1 오퍼랜드와 제 2 오퍼랜드와 논리합한 결과를 저장합니다.

(ORMV 의 사용 예)



- 상위 래더 다이어그램은 입력 접점 중 어드레스 X2 의 32 비트 데이터에 하위 5 번째 비트를 논리합하여 어드레스 R2 로 출력하기 위해 작성한 부분입니다.
- 논리합 데이터 오퍼랜드 OP2 는 X,Y,R,G,F,D,T,C,정수를 사용할 수 있으며, 예에서는 정수를 사용하였습니다. PLC EDITOR 에서 논리곱 데이터 입력 시 ‘H’를 사용하면 16 진수로 입력이 가능합니다.  
(‘H’를 사용하지 않은 경우는 정수 데이터가 됩니다.)
- 위 예에서는 하위 5 번째 비트를 논리합 하기 위해 ‘H10’을 입력하였습니다.

6.8) DSCHB ( SUB 34 : 바이너리 데이터 테이블 검색 )

[명령어: SUB 34 DSCHB]

(기능)

데이터 테이블내의 데이터를 검색하는 명령입니다.

검색데이터 어드레스의 데이터를 데이터테이블(D-어드레스)에서 검색한 후 일치하는 테이블 번호를 검색결과 출력 어드레스에 출력합니다.

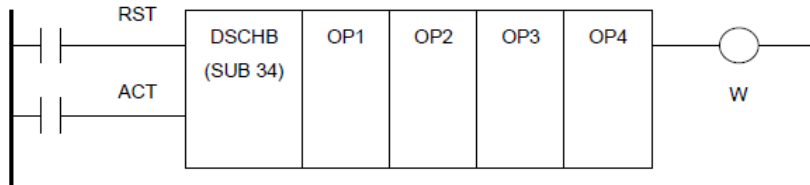
(형식)

| DSCHB | OP1                | OP2 | OP3 | OP4 | OP5 |
|-------|--------------------|-----|-----|-----|-----|
| OP1   | 데이터테이블의 SIZE 설정    |     |     |     |     |
| OP2   | 데이터테이블의 선두 어드레스 설정 |     |     |     |     |
| OP3   | 검색데이터 설정           |     |     |     |     |
| OP4   | 검색결과 출력 어드레스       |     |     |     |     |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|----------------------------|---|---|---|---|---|---|---|-----|
|      | X                          | Y | R | G | F | D | T | C | INT |
| OP1  | O                          | O | O | O | O | O | X | O | X   |
| OP2  | O                          | O | O | O | O | O | X | O | X   |
| OP3  | O                          | O | O | O | O | O | X | O | X   |
| OP4  | X                          | O | O | O | X | O | X | O | X   |

(래더)



(설명)

## 1) 제어조건

### a) 리세트 : RST

RST = 0 : 리세트 해제

RST = 1 : 리세트, 출력(W)은 0 이 됩니다.

### b) 실행지령 : ACT

ACT = 0 : DSCHB 명령을 실행하지 않습니다. 출력 W 는 변화하지 않습니다.

ACT = 1 : DSCHB 명령을 실행합니다.

## 2) 제 1 오퍼랜드(OP1) : 데이터 테이블의 데이터 SIZE 설정

데이터 테이블의 길이를 설정합니다.

즉 data table 의 선두를 0 번으로 하여 data table 의 최후가 n 번인 경우,  
n+1 을 data table 의 data 수로 설정합니다.

## 제 2 오퍼랜드(OP2) : 데이터테이블 선두 어드레스 설정

데이터테이블의 선두 어드레스를 설정합니다.

설정된 값이 데이터 테이블의 선두 D 어드레스를 의미합니다.

## 제 3 오퍼랜드(OP3) : 검색 데이터 어드레스 설정

검색하고자 하는 데이터가 설정되어있는 어드레스입니다.

## 제 4 오퍼랜드(OP4) : 검색결과 출력 어드레스

데이터가 검색되면 그 데이터를 저장하고 있는 테이블의 번호를 설정된 어드레스로 출력합니다.

여기서, 검색된 데이터 테이블의 번호는 0 ~ n 가 됩니다.

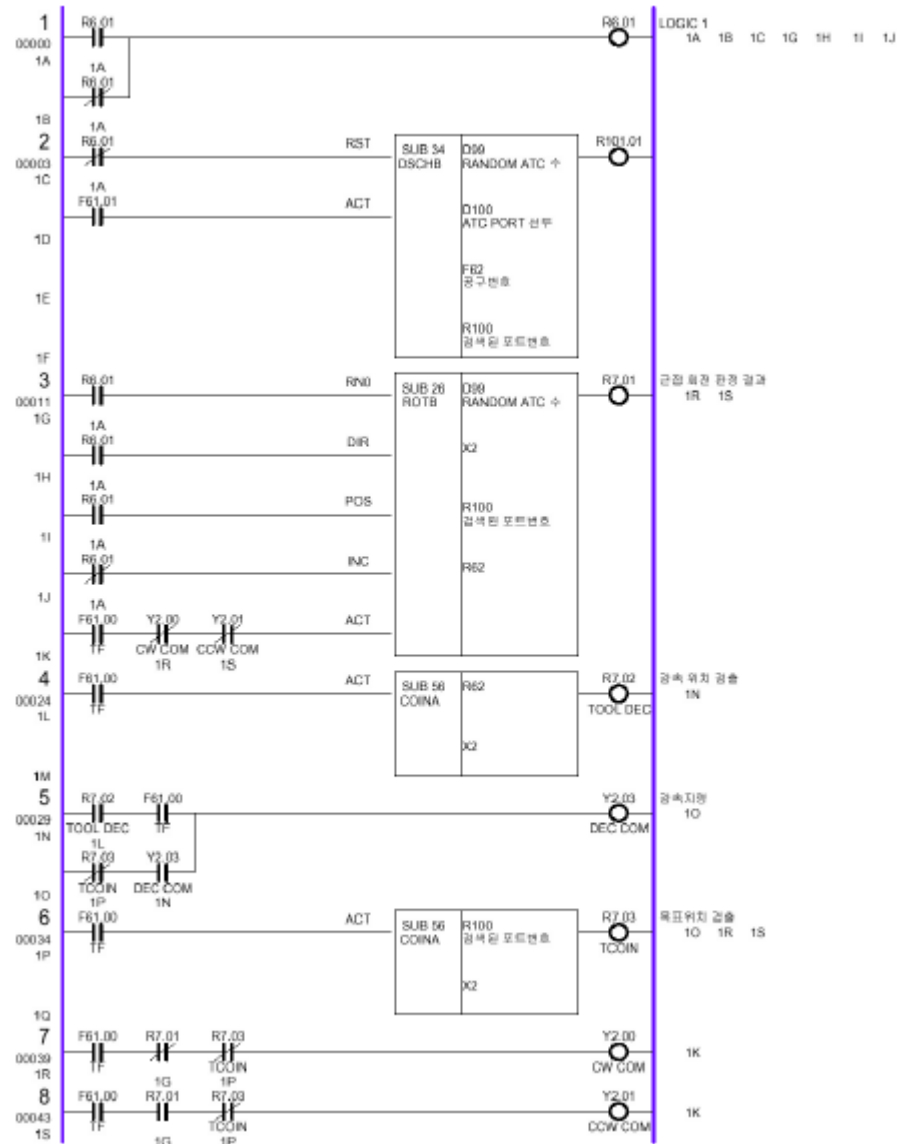
결과 출력 : W

W = 0 : 검색 데이터가 데이터 테이블 내에 있음

W = 1 : 검색 데이터가 데이터 테이블 내에 없음

(DSCHB 사용 예)

- 다음 PAGE 에는 랜덤 공구 교환 시 공구번호에 해당하는 포트 번호를 검색하는 부분이 DSCHB 명령어를 사용하여 래더 다이어그램이 작성되어 있습니다.
- 랜덤 방식의 ATC 의 포트 개수+1 가 어드레스 D99 에 설정되어 있으며, 데이터 테이블의 선두 어드레스는 D100 에 설정되어 있습니다. 예], D99 = '11', D100 = '10'으로 한 경우, 데이터 테이블의 선두 어드레스는 D10 이 되며 1 번 포트는 D11, 2 번 포트는 D12 순으로 포트들에 저장되어 있는 공구 번호가 기억되게 됩니다.
- T 코드 개시 신호(TF) F61.0 에 의해 DSCHB 명령어가 수행되어 T Code 번호 데이터(어드레스 F62)를 어드레스 D10 부터 검색을 진행하게 됩니다. 검색 데이터가 존재하게 되면 어드레스 R100 에 포트번호가 출력됩니다. 그렇지 않은 경우 출력 어드레스 R101.1 이 '1'이 됩니다.

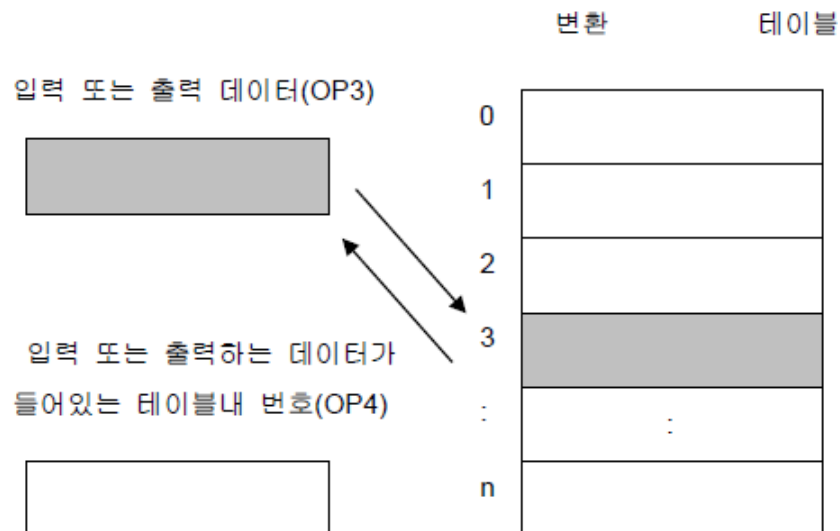


## 6.9) XMOVB ( SUB 35: 바이너리 인덱스 수식 데이터 전송 )

[명령어: SUB 35 XMOVB]

(기능)

DSCHB 명령과 같은 형식으로 이 명령도 데이터 테이블과 연관된 명령어입니다. 데이터 테이블의 내용(데이터)을 읽거나, 바꿔 적어 넣기를 하는 명령입니다. 여기서 데이터 테이블은 D 어드레스를 의미합니다.



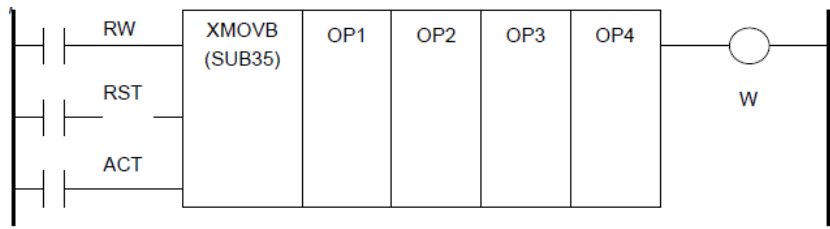
(형식)

|     | XMOVB                    | OP1 | OP2 | OP3 | OP4 |
|-----|--------------------------|-----|-----|-----|-----|
| OP1 | 데이터테이블의 데이터 개수 저장 (SIZE) |     |     |     |     |
| OP2 | 데이터테이블의 선두 어드레스 설정       |     |     |     |     |
| OP3 | 입출력 데이터 저장 어드레스          |     |     |     |     |
| OP4 | 테이블내 저장 번호 어드레스          |     |     |     |     |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   | 정수 |
|------|----------------------------|---|---|---|---|---|---|---|----|
|      | X                          | Y | R | G | F | D | T | C |    |
| OP1  | O                          | O | O | O | O | O | X | X | X  |
| OP2  | O                          | O | O | O | O | O | X | X | X  |
| OP3  | O                          | O | O | O | O | O | X | X | X  |
| OP4  | X                          | O | O | O | X | O | X | X | X  |

(래더)



(설명)

1) 제어조건

a) READ, WRITE 지정 : RW

RW = 0 : 데이터테이블에서 데이터를 읽음

RW = 1 : 데이터테이블로 데이터를 적어 넣음

b) 리세트 : RST

RST = 0 : 리세트 해제

RST = 1 : 리세트, 출력(W)은 0 이 됩니다.

c) 실행지령 : ACT

ACT = 0 : XMOVB 명령을 실행하지 않습니다. 출력 W 는 변화하지 않습니다.

ACT = 1 : XMOVB 명령을 실행합니다.

2) 제 1 오퍼랜드(OP1) : 데이터 테이블의 데이터 수 저장

데이터 테이블의 길이를 설정합니다.

즉 data table 의 선두를 0 번으로 하여 data table 의 최후가 n 번인 경우,  
n+1 을 data table 의 data 수로 설정합니다.

제 2 오퍼랜드(OP2) : 데이터 테이블 선두 어드레스

데이터테이블의 선두 어드레스를 설정합니다.

설정된 값이 데이터 테이블의 선두 D 어드레스를 의미합니다.

제 3 오퍼랜드(OP3) : 입출력 데이터 저장 어드레스

데이터 테이블로 저장할 데이터를 가지고 있는 어드레스도 되고,  
데이터 테이블로부터 읽은 데이터를 저장하는 어드레스입니다.

SR2(Stack Register 3 번째 bit)가 0 인 경우 OP3 어드레스에 OP2+OP4 어드레스의 값을 저장합니다.

SR2 가 1 인 경우 OP3 의 값을 OP2+OP4 의 출력 어드레스에 저장합니다.

제 4 오퍼랜드(OP4) : 테이블 내의 번호 저장 어드레스

테이블 내의 번호를 저장하는 어드레스입니다.

OP1 의 수보다 크게 설정한 경우 W=1(ERROR)이 출력됩니다.

결과 출력 : W

W = 0 : ERROR 없음

W = 1 : ERROR 있음 ( 지정한 TABLE 번호가 데이터 테이블의 크기를 넘어가는 경우 )

[XMOVEB 사용 예]

만약, D99 = '5', D100 = '10'이라면

→ 데이터 테이블은 D10(D100의 값)을 선두 어드레스로 하여 D10~D14 구성됩니다.

예 1] R31 = '3', R100.00 = '0'인 경우는

R100.00 = '0'이므로 D(OP2+'R31')의 데이터, 즉 D13의 데이터를 OP3인 R30에 저장합니다.

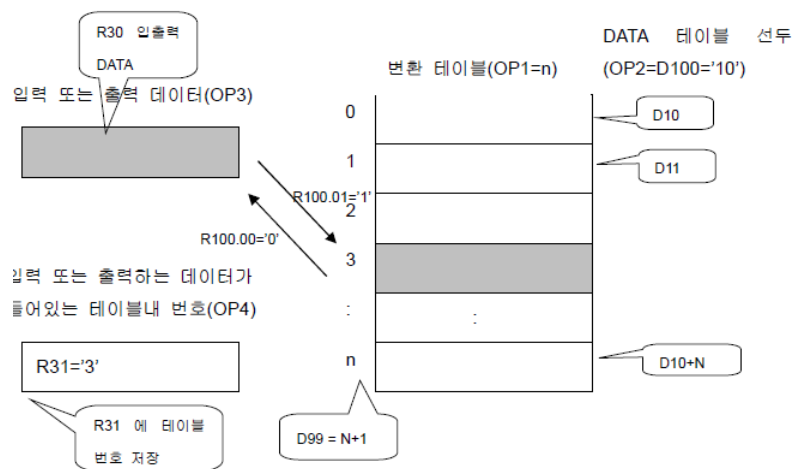
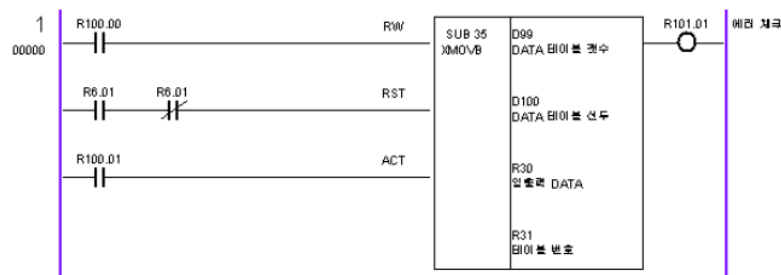
예 2] R30 = '100', R100.00 = '1'인 경우

R100.00 = '1'이므로 OP3인 R30의 데이터를 D(OP2+'R31'), 즉 어드레스 D13에 저장합니다.

예 3] R31 = '10', R100.00 = '0'인 경우는

D(OP2+'R31'), 즉 어드레스 D20은 데이터 테이블(D10~D14) 영역을 벗어나므로

R101.01은 HIGH가 되어 알람으로 처리해야 됩니다.



6.10) ADDB ( SUB 36 : 바이너리 가산 )

[명령어: SUB 36    ADDB]

(기능)

BINARY 형식 데이터 가산을 합니다.

(형식)

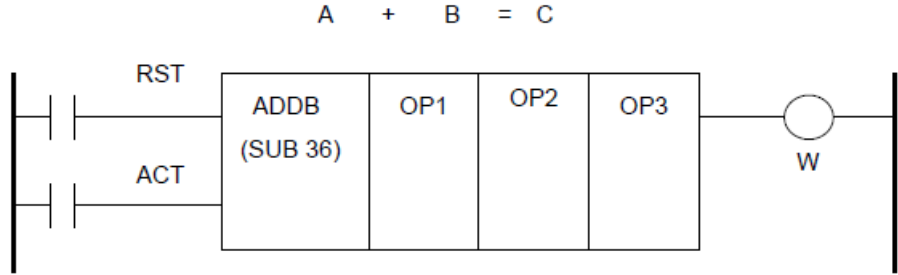
| ADDB    OP1    OP2    OP3 |               |  |  |
|---------------------------|---------------|--|--|
| OP1                       | 피가수 데이터 오퍼랜드  |  |  |
| OP2                       | 가수 데이터 오퍼랜드   |  |  |
| OP3                       | 가산 결과 출력 어드레스 |  |  |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능    X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|-------------------------------|---|---|---|---|---|---|---|-----|
|      | X                             | Y | R | G | F | D | T | C | INT |
| OP1  | O                             | O | O | O | O | O | X | O | O   |
| OP2  | O                             | O | O | O | O | O | X | O | O   |
| OP3  | X                             | O | O | O | X | O | X | O | X   |

| 명령어  | OP1  | OP2  | OP3  |
|------|------|------|------|
| ADDB | 어드레스 | 어드레스 | 어드레스 |
|      | 어드레스 | 상수   | 어드레스 |
|      | 상수   | 어드레스 | 어드레스 |
|      | 상수   | 상수   | 어드레스 |

(래더)



(설명)

1) 제어조건

a) 리세트 : RST

RST = 0 : 리세트 해제

RST = 1 : 리세트, 출력(W)은 0 이 됩니다.

b) 실행지령 : ACT

ACT = 0 : ADDB 명령을 실행하지 않습니다. 출력 W 는 변화하지 않습니다.

ACT = 1 : ADDB 명령을 실행합니다.

2) 제 1 오퍼랜드(OP1) : 피가수데이터 오퍼랜드

피가수데이터 오퍼랜드입니다.

제 2 오퍼랜드(OP2) : 가수데이터 오퍼랜드

가수데이터 오퍼랜드입니다.

제 3 오퍼랜드(OP3) : 검색결과 출력 어드레스

가산한 결과가 출력되는 어드레스를 지정합니다.

결과 출력 : W

W = 0 : 연산 정상

W = 1 : 연산 이상. 가산 결과가 지정 DATA 길이를 넘은 경우 W=1 로 됩니다.

## 6.11) SUBB ( SUB 37 : 바이너리 감산 )

[명령어: SUB 37 SUBB]

(기능)

BINARY 형식 데이터 감산을 합니다.

(형식)

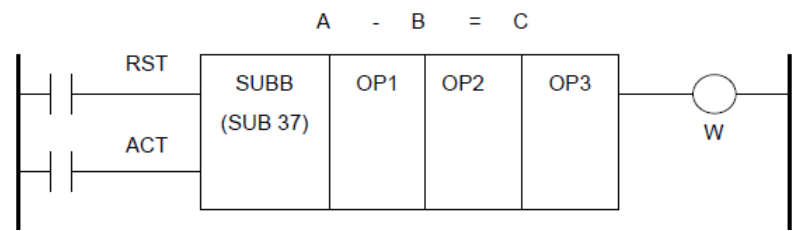
|     | SUBB | OP1           | OP2 | OP3 |
|-----|------|---------------|-----|-----|
| OP1 |      | 피감수 데이터 오퍼랜드  |     |     |
| OP2 |      | 감수 데이터 오퍼랜드   |     |     |
| OP3 |      | 감산 결과 출력 어드레스 |     |     |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|----------------------------|---|---|---|---|---|---|---|-----|
|      | X                          | Y | R | G | F | D | T | C | INT |
| OP1  | O                          | O | O | O | O | O | X | O | O   |
| OP2  | O                          | O | O | O | O | O | X | O | O   |
| OP3  | X                          | O | O | O | X | O | X | O | X   |

| 명령어  | OP1  | OP2  | OP3  |
|------|------|------|------|
| SUBB | 어드레스 | 어드레스 | 어드레스 |
|      | 어드레스 | 상수   | 어드레스 |
|      | 상수   | 어드레스 | 어드레스 |
|      | 상수   | 상수   | 어드레스 |

(래더)



(설명)

- 1) 제어조건
  - a) 리세트 : RST
  - RST = 0 : 리세트 해제
  - RST = 1 : 리세트, 출력(W)은 0 이 됩니다.
  - b) 실행지령 : ACT
  - ACT = 0 : SUBB 명령을 실행하지 않는다. 출력 W 는 변화하지 않습니다.
  - ACT = 1 : SUBB 명령을 실행합니다.
- 2) 제 1 오퍼랜드(OP1) : 피감수데이터 오퍼랜드
- 피감수데이터 오퍼랜드입니다.
- 제 2 오퍼랜드(OP2) : 감수데이터 오퍼랜드
- 감수데이터 오퍼랜드입니다.
- 제 3 오퍼랜드(OP3) : 감산결과 출력 어드레스
- 감산 결과가 출력되는 어드레스를 지정합니다.
- 결과 출력 : W
- W = 0 : 연산 정상
- W = 1 : 연산 이상. 감산 결과가 지정 DATA 길이를 넘은 경우 W=1 로 됩니다.

6.12) MULB ( SUB 38 : 바이너리 승산 )

[ 령어: SUB 38MULB]

(기능)

BINARY 형식 데이터 승산을 합니다.

(형식)

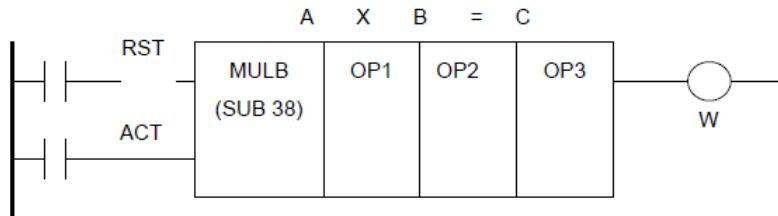
| MULB |  | OP1           | OP2 | OP3 |
|------|--|---------------|-----|-----|
| OP1  |  | 피승수 데이터 오퍼랜드  |     |     |
| OP2  |  | 승수 데이터 오퍼랜드   |     |     |
| OP3  |  | 승산 결과 출력 어드레스 |     |     |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   | 정수 |
|------|----------------------------|---|---|---|---|---|---|---|----|
|      | X                          | Y | R | G | F | D | T | C |    |
| OP1  | O                          | O | O | O | O | O | X | O | O  |
| OP2  | O                          | O | O | O | O | O | X | O | O  |
| OP3  | X                          | O | O | O | X | O | X | O | X  |

| 명령어  | OP1  | OP2  | OP3  |
|------|------|------|------|
| MULB | 어드레스 | 어드레스 | 어드레스 |
|      | 어드레스 | 상수   | 어드레스 |
|      | 상수   | 어드레스 | 어드레스 |
|      | 상수   | 상수   | 어드레스 |

(래더)



(설명)

## 1) 제어조건

### a) 리세트 : RST

RST = 0 : 리세트 해제

RST = 1 : 리세트, 출력(W)은 0 이 됩니다.

### b) 실행지령 : ACT

ACT = 0 : MULB 명령을 실행하지 않습니다. 출력 W 는 변화하지 않습니다.

ACT = 1 : MULB 명령을 실행합니다.

## 2) 제 1 오퍼랜드(OP1) : 피승수데이터 오퍼랜드

피승수데이터 오퍼랜드입니다.

## 제 2 오퍼랜드(OP2) : 승수데이터 오퍼랜드

승수데이터 오퍼랜드입니다.

## 제 3 오퍼랜드(OP3) : 연산결과 출력 어드레스

승산 결과가 출력되는 어드레스를 지정합니다.

결과 출력 : W

W = 0 : 연산 정상

W = 1 : 연산 이상. 승산 결과가 지정 DATA 길이를 넘은 경우 W=1 로 됩니다.

6.13) DIVB ( SUB 39 : 바이너리 제산 )

[명령어: SUB 39 DIVB]

(기능)

BINARY 데이터 제산을 합니다.

(형식)

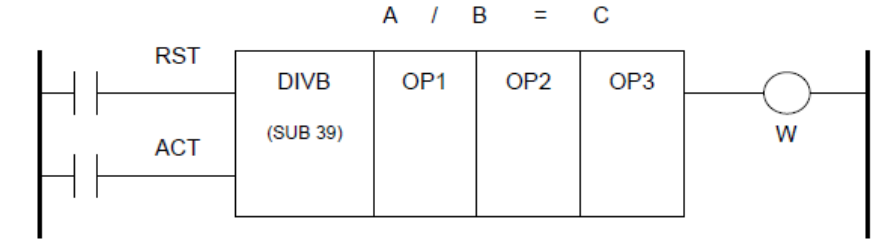
|     | DIVB          | OP1 | OP2 | OP3 |
|-----|---------------|-----|-----|-----|
| OP1 | 피제수 데이터 오퍼랜드  |     |     |     |
| OP2 | 제수 데이터 오퍼랜드   |     |     |     |
| OP3 | 제산 결과 출력 어드레스 |     |     |     |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|----------------------------|---|---|---|---|---|---|---|-----|
|      | X                          | Y | R | G | F | D | T | C | INT |
| OP1  | O                          | O | O | O | O | O | X | O | O   |
| OP2  | O                          | O | O | O | O | O | X | O | O   |
| OP3  | X                          | O | O | O | X | O | X | O | X   |

| 명령어  | OP1  | OP2  | OP3  |
|------|------|------|------|
| DIVB | 어드레스 | 어드레스 | 어드레스 |
|      | 어드레스 | 상수   | 어드레스 |
|      | 상수   | 어드레스 | 어드레스 |
|      | 상수   | 상수   | 어드레스 |

(래더)



(설명)

1) 제어조건

a) 리세트 : RST

RST = 0 : 리세트 해제

RST = 1 : 리세트, 출력(W)은 0 이 됩니다.

b) 실행지령 : ACT

ACT = 0 : DIVB 명령을 실행하지 않습니다. 출력 W 는 변화하지 않습니다.

ACT = 1 : DIVB 명령을 실행합니다.

2) 제 1 오퍼랜드(OP1) : 피제수데이터 오퍼랜드

피제산데이터 오퍼랜드입니다.

제 2 오퍼랜드(OP2) : 제수데이터 오퍼랜드

제산데이터 오퍼랜드입니다.

제 3 오퍼랜드(OP3) : 제산 결과 출력 어드레스

제산결과가 출력되는 어드레스를 지정합니다.

결과 출력 : W

W=0 : 연산 정상

W=1 : 연산 이상. 제산 결과가 지정 DATA 길이를 넘은 경우 W=1 로 됩니다.

## 6.14) MOVB ( SUB 47 : 데이터 전송 )

[명령어: SUB 47 MOVB]

(기능)

오퍼랜드 1 의 값을 오퍼랜드 2 의 어드레스로 전송합니다.

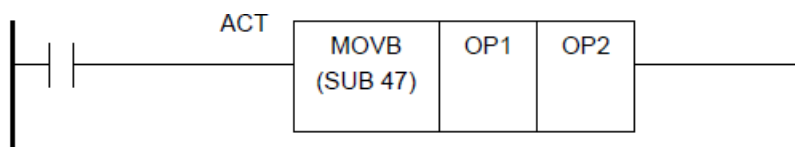
(형식)

|     | MOVB | OP1         | OP2 |
|-----|------|-------------|-----|
| OP1 |      | 입력 데이터 오퍼랜드 |     |
| OP2 |      | 출력 데이터 어드레스 |     |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|----------------------------|---|---|---|---|---|---|---|-----|
|      | X                          | Y | R | G | F | D | T | C | INT |
| OP1  | O                          | O | O | O | O | O | X | O | O   |
| OP2  | X                          | O | O | O | X | O | X | O | X   |

(래더)



(설명)

1) 제어조건

a) 실행지령 : ACT

ACT=0 : MOVB 명령어를 실행하지 않습니다.

ACT=1 : MOVB 명령어를 실행합니다.

2) 제 1 오퍼랜드(OP1) : 입력 데이터 오퍼랜드

전송되어질 데이터를 지정합니다.

제 2 오퍼랜드(OP2) : 출력 데이터 어드레스

제 1 오퍼랜드의 데이터를 저장하는 어드레스입니다.

6.15) CTRA ( SUB 55 : 카운터 )

[명령어: SUB 55 CTRA]

(기능)

링 카운터이면서 업다운 카운터 기능을 합니다. Counter 의 일반적인 사용방법은 수리 적산입니다.  
그러나 공작기계에서 counter 는 여러 가지 용도에 이용됩니다.  
이 counter 의 수치 data(preset 값, 적산치)는 정수 형식입니다.

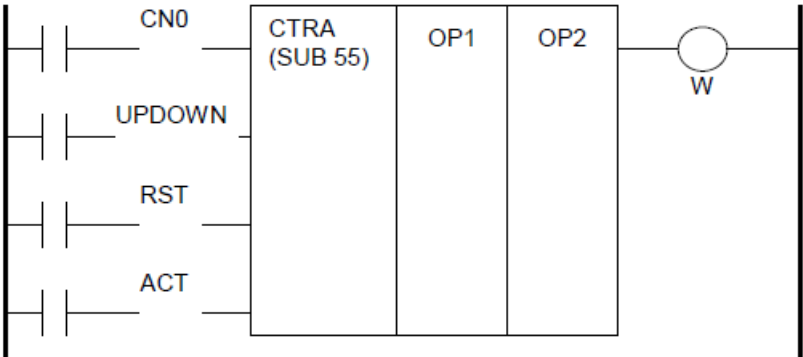
(형식)

| CTRA      OP1      OP2 |                   |
|------------------------|-------------------|
| OP1                    | COUNTER 레지스터 어드레스 |
| OP2                    | COUNTER PRESET 값  |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능    X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|-------------------------------|---|---|---|---|---|---|---|-----|
|      | X                             | Y | R | G | F | D | T | C | INT |
| OP1  | X                             | X | X | X | X | X | X | O | X   |
| OP2  | X                             | X | X | X | X | O | X | X | O   |

(래더)



(설명)

- 1) 제어조건
  - a) 초기치 설정 : CN0  
CN0 = 0 : 카운터의 적산치가 0 부터 시작.      0,1,2,3, ...,N  
CN0 = 0 → 1 : 카운터의 적산치가 1 부터 시작. 1,2,3, ...,N (0 은 사용하지 않습니다.)
  - b) 업다운카운터 지정 : UPDOWN  
UPDOWN = 0 : UP 카운터, 적산치의 초기값은 CN0 에 따라 0 또는 1 이 됩니다.  
UPDOWN = 1 : DOWN 카운터, 적산치의 초기값은 프리셋 값이 됩니다.
  - c) 리세트 : RST  
RST = 0 : 리세트 해제  
RST = 1 : 리세트, 출력(W)은 0 이 되고, 적산치는 초기치로 reset 됩니다.

## d) 카운트 신호 : ACT

ACT=0 : 카운트 동작을 하지 않습니다. 출력 변화가 없습니다.

ACT=1 : ACT 값이 0 에서 1 로 변하는 경우에만 카운트 동작을 행합니다.

### 2) 제 1 오퍼랜드(OP1) : 카운터 번호

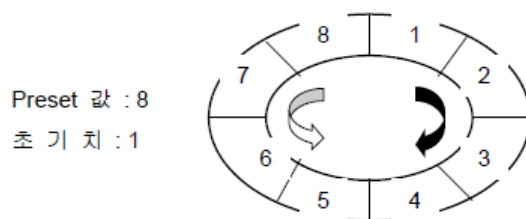
C 어드레스만 가능

제 2 오퍼랜드(OP2) : 카운터 프리셋(preset) 값(= counter 제어 data address)

Counter 제어 data 의 선두 address 를 설정합니다.

D 어드레스와 정수타입만 가능

상수값의 범위 : 1 ~ 2147483647



결과 출력: W

OP2 에 의해 지정된 PRESET 값(설정치)에 도달하면 W=1 이 됩니다.

Counter 의 사용 예)

Work 의 가공 수를 counter 하고 먼저 설정한 수로 되면 신호를 출력합니다.

회전체의 위치를 기억하기 위해 counter 를 사용합니다.

## 6.16) COINA ( SUB 56 : 일치판정 )

[명령어: SUB 56 COINA] : 800S 의 DEC 명령과 같음

(기능)

두 오퍼랜드의 값이 일치하는지 판정합니다.

(형식)

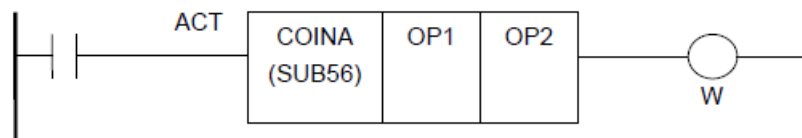
|     | COINA | OP1            | OP2 |
|-----|-------|----------------|-----|
| OP1 |       | 일치비교 대상 오퍼랜드 1 |     |
| OP2 |       | 일치비교 대상 오퍼랜드 2 |     |

| 명령어   | OP1  | OP2  |
|-------|------|------|
| COINA | 어드레스 | 어드레스 |
|       | 어드레스 | 상수   |
|       | 상수   | 어드레스 |

(사용가능 어드레스)

[illegible]

(래더)



(설명)

- ### 1) 제어조건

ACT=0 : 일치판정 결과 출력(W)을 OFF 합니다.

**ACT=1 : 일치판정 처리를 합니다.**

- 2) 제 1 오퍼랜드 : 일치비교 대상 오퍼랜드 1

어드레스인 경우는 비교 대상이 어드레스 번호의 값이 되고,

정수인 경우는 그 값 자체가 비교 대상입니다.

제 2 오퍼랜드 : 일치비교 대상 오퍼랜드 2

어드레스인 경우는 비교 대상이 어드레스 번호의 값이 되고,

정수인 경우는 그 값 자체가 비교 대상입니다.

결과 출력 : W

제 1 오퍼랜드와 제 2 오퍼랜드의 값이 같으면 1, 다르면 0을 출력합니다.

### 6.17) CALL ( SUB 65 : 서브루틴 콜)

[명령어: SUB65 CALL]

(기능)

서브루틴을 CALL 하는 기능을 수행합니다.

S 어드레스 번호로 시작하는 SP 명령부터 SPE 명령 사이의 시퀀스 프로그램을 실행합니다.

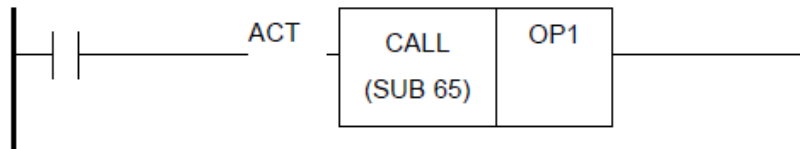
(형식)

|     |               |            |
|-----|---------------|------------|
|     | <b>CALL</b>   | <b>OP1</b> |
| OP1 | 서브루틴 인덱스 어드레스 |            |

(사용가능 어드레스)

[illegible]

(래더)



(설명)

- 1) 제어조건
    - a) 실행명령 : ACT
- ACT = 0 : CALL 명령을 실행하지 않습니다.  
 ACT = 1 : CALL 명령을 실행합니다.
- 2) 제 1 오퍼랜드(OP1) : 서브루틴 인덱스 어드레스
- S 어드레스만 사용 가능.

## 6.18) JMPB ( SUB 68 : 점프처리 )

[명령어: SUB68 JMPB]

(기능)

L 어드레스 번호로 시작하는 LABEL 명령어로 분기합니다.

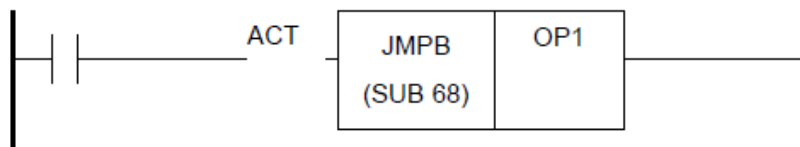
(형식)

| JMPB OP1 |              |
|----------|--------------|
| OP1      | 레이블 인덱스 어드레스 |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   |     |
|------|----------------------------|---|---|---|---|---|---|---|-----|
|      | X                          | Y | R | G | F | D | T | C | INT |
| OP1  | X                          | X | X | X | X | X | X | X | X   |

(래더)



(설명)

- 1) 제어조건
    - a) 실행명령 : ACT
- ACT = 0 : JMPB 명령을 실행하지 않습니다.  
 ACT = 1 : JMPB 명령을 실행합니다.
- 2) 제 1 오퍼랜드(OP1) : 레이블 인덱스 어드레스
- L 어드레스만 사용 가능.

**6.19) LBL ( SUB 69 : 레이블표시 )**

[명령어: SUB69 LBL]

(기능)

L 어드레스 번호로 시작하는 LABEL 을 나타냅니다.

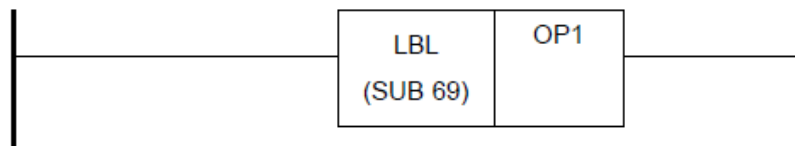
(형식)

| LBL      OP1 |              |
|--------------|--------------|
| OP1          | 레이블 인덱스 어드레스 |

(사용가능 어드레스)

[illegible]

(래더)



(설명)

- 1) 제 1 오퍼랜드(OP1): 레이블 인덱스 어드레스  
L 어드레스만 사용 가능.

**6.20) SP ( SUB 71 : 서브루틴 시작 )**

[ 명령어: SUB71 SP ]

(기능)

S 어드레스 번호의 서브루틴 시작을 나타냅니다.

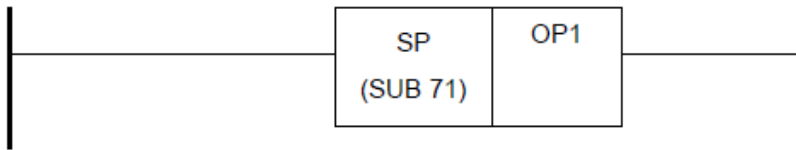
(형식)

|     |               |     |
|-----|---------------|-----|
| SP  |               | OP1 |
| OP1 | 서브루틴 인덱스 어드레스 |     |

(사용가능 어드레스)

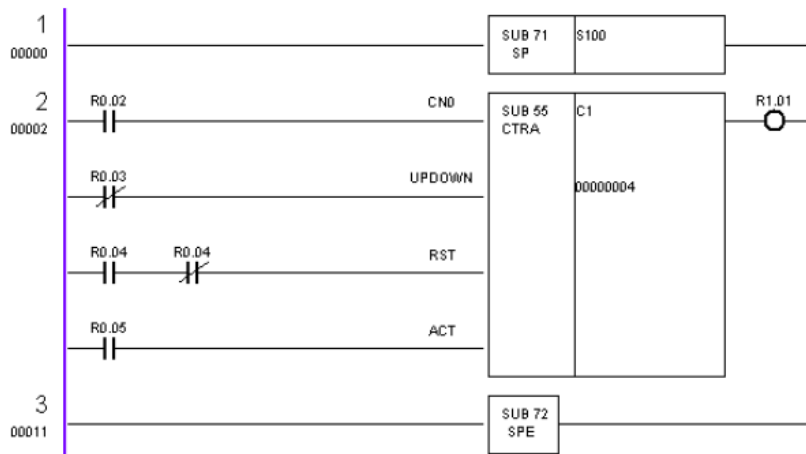
[illegible]

(래더)



(설명)

- 1) 제 1 오퍼랜드(OP1) : 서브루틴 인덱스 어드레스  
S 어드레스만 사용 가능.



## 6.21) SPE ( SUB 72 : 서브루틴 종료 )

[ 령어: SUB 72 SPE ]

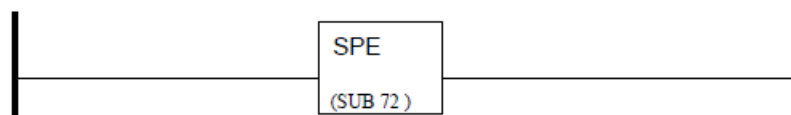
(기능)

이 명령은 SP 명령어와 함께 쓰이며, 단독으로는 사용되어질 수 없습니다.  
서브루틴의 종료를 의미합니다.

(형식)

SPE

(래더)



6.22) INV ( SUB 74 : 비트 반전 처리 )

[명령어: SUB74 INV]

(기능)

비트 반전 대상 어드레스의 비트 값을 반전시킵니다.

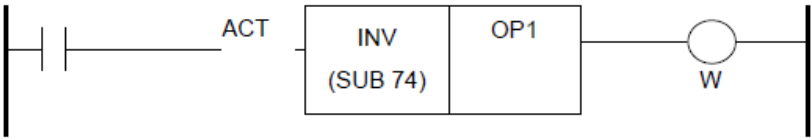
(형식)

| INV          OP1 |               |
|------------------|---------------|
| OP1              | 비트 반전 대상 어드레스 |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능    X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|-------------------------------|---|---|---|---|---|---|---|-----|
|      | X                             | Y | R | G | F | D | T | C | INT |
| OP1  | O                             | O | O | O | O | X | X | X | X   |

(래더)



(설명)

- 1) 제어조건
  - a) 실행명령 : ACT
  - ACT = 0 : INV 명령을 실행하지 않습니다. (W는 이전 결과를 유지합니다.)
  - ACT = 1 : INV 명령을 실행합니다.
- 2) 제 1 오퍼랜드(OP1) : 비트 반전 대상 어드레스의 비트
  - 예) R10.00 은 어드레스 R10 의 1 번째 비트 (W(R10 을 쓰면 R10.00 으로 처리됨)
- 3) 결과출력 : W
  - W = 0 : OP1 의 해당 비트가 HIGH 인 경우에 W 는 0 이 됩니다.
  - W = 1 : OP1 의 해당 비트가 LOW 인 경우에 W 는 1 이 됩니다.

## 6.23) EQU ( SUB 75: 동등비교 )

[명령어: SUB 75 EQU]

(기능)

오퍼랜드 1 과 오퍼랜드 2 를 비교합니다.

(형식)

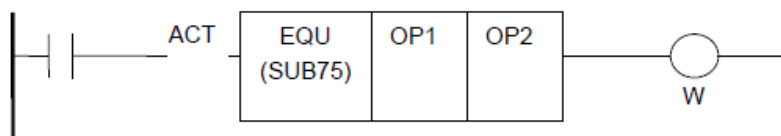
|     | EQU | OP1          | OP2 |
|-----|-----|--------------|-----|
| OP1 |     | 비교 대상 오퍼랜드 1 |     |
| OP2 |     | 비교 대상 오퍼랜드 2 |     |

| 명령어 | OP1  | OP2  |
|-----|------|------|
| EQU | 어드레스 | 어드레스 |
|     | 어드레스 | 상수   |
|     | 상수   | 어드레스 |
|     | 상수   | 상수   |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|----------------------------|---|---|---|---|---|---|---|-----|
|      | X                          | Y | R | G | F | D | T | C | INT |
| OP1  | O                          | O | O | O | O | O | O | O | O   |
| OP2  | O                          | O | O | O | O | O | O | O | O   |

(래더)



(설명)

### 1) 제어조건

#### a) 실행명령 : ACT

ACT = 0 : EQU 명령을 실행하지 않습니다. (W는 이전 결과를 유지합니다.)

ACT = 1 : EQU 명령을 실행합니다.

제 1 오퍼랜드(OP1) : 비교 대상 오퍼랜드 1

제 2 오퍼랜드(OP2) : 비교 대상 오퍼랜드 2

결과출력 : W

W = 0 : OP1 과 OP2 의 값이 같지 않은 경우에 W 는 0 이 됩니다.

W = 1 : OP1 과 OP2 의 값이 같은 경우에 W 는 1 이 됩니다.

6.24) GT ( SUB 76: GREATER THEN )

[명령어: SUB 76 GT]

(기능)

오퍼랜드 1 과 오퍼랜드 2 를 비교합니다.

(형식)

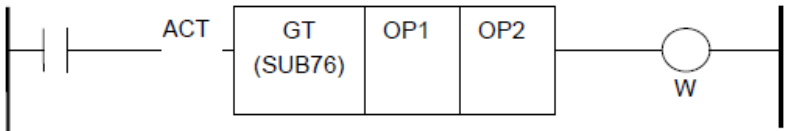
| GT      OP1    OP2 |              |
|--------------------|--------------|
| OP1                | 비교 대상 오퍼랜드 1 |
| OP2                | 비교 대상 오퍼랜드 2 |

| 명령어 | OP1  | OP2  |
|-----|------|------|
| GT  | 어드레스 | 어드레스 |
|     | 어드레스 | 상수   |
|     | 상수   | 어드레스 |
|     | 상수   | 상수   |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능    X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|-------------------------------|---|---|---|---|---|---|---|-----|
|      | X                             | Y | R | G | F | D | T | C | INT |
| OP1  | O                             | O | O | O | O | O | O | O | O   |
| OP2  | O                             | O | O | O | O | O | O | O | O   |

(래더)



(설명)

- 1) 제어조건
- a) 실행명령 :ACT
- ACT = 0 : GT 명령을 실행하지 않습니다. (W는 이전 결과를 유지합니다.)
- ACT = 1 : GT 명령을 실행합니다.
- 2) 제 1 오퍼랜드(OP1) : 비교 대상 오퍼랜드 1
- 3) 제 2 오퍼랜드(OP2) : 비교 대상 오퍼랜드 2
- 4) 결과출력 : W
- W = 0 : OP1 <= OP2 의 경우에 W 는 0 이 됩니다.
- W = 1 : OP1 > OP2 의 경우에 W 는 1 이 됩니다.

## 6.25) LT ( SUB 77: LESS THEN )

[명령어: SUB 77 LT]

(기능)

오퍼랜드 1 과 오퍼랜드 2 를 비교합니다.

(형식)

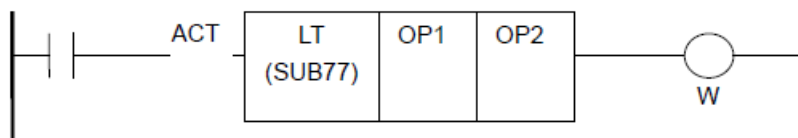
| LT  | OP1          | OP2 |
|-----|--------------|-----|
| OP1 | 비교 대상 오퍼랜드 1 |     |
| OP2 | 비교 대상 오퍼랜드 2 |     |

| 명령어 | OP1  | OP2  |
|-----|------|------|
| LT  | 어드레스 | 어드레스 |
|     | 어드레스 | 상수   |
|     | 상수   | 어드레스 |
|     | 상수   | 상수   |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   |     |
|------|----------------------------|---|---|---|---|---|---|---|-----|
|      | X                          | Y | R | G | F | D | T | C | INT |
| OP1  | O                          | O | O | O | O | O | O | O | O   |
| OP2  | O                          | O | O | O | O | O | O | O | O   |

(래더)



(설명)

- 1) 제어조건
- 2) 실행명령 : ACT
- 3) ACT = 0 : LT 명령을 실행하지 않습니다. (W는 이전 결과를 유지합니다.)
- 4) ACT = 1 : LT 명령을 실행합니다.
- 5) 제 1 오퍼랜드(OP1) : 비교 대상 오퍼랜드 1
- 6) 제 2 오퍼랜드(OP2) : 비교 대상 오퍼랜드 2
- 7) 결과출력 : W
- 8) W = 0 : OP1 >= OP2 의 경우에 W 는 0 이 됩니다.
- 9) W = 1 : OP1 < OP2 의 경우에 W 는 1 이 됩니다.

6.26) SFR (SUB 78: SHIFT RIGHT )

[명령어: SUB 78 SFR]

(기능)

Shift 대상 어드레스(OP1)에 있는 데이터를 Shift 비트 오퍼랜드(OP2)에 지정된 수만큼 우측으로 SHIFT 합니다.

또한 SHR 은 산술 쉬프트(Arithmetic Shift)를 합니다.

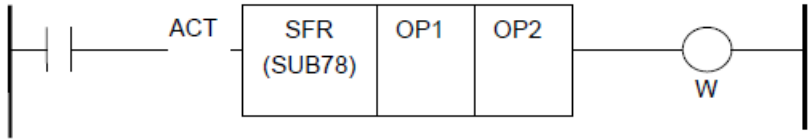
(형식)

| SFR      OP1    OP2 |                 |
|---------------------|-----------------|
| OP1                 | 쉬프트 대상 어드레스     |
| OP2                 | 쉬프트 비트 데이터 어드레스 |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능    X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|-------------------------------|---|---|---|---|---|---|---|-----|
|      | X                             | Y | R | G | F | D | T | C | INT |
| OP1  | X                             | O | O | O | X | O | X | O | X   |
| OP2  | X                             | O | O | O | X | O | X | O | O   |

(래더)



(설명)

- 1) 제어조건
  - a) 실행지령 :ACT  
ACT = 0 : SFR 명령을 실행하지 않습니다.  
ACT = 펄스 신호 : SFR 명령을 실행합니다.  
(주) 반드시 ACT=1 에서 명령 실행한 후, 바로 0 으로 해야만 합니다.
- 2) 제 1 오퍼랜드(OP1): 쉬프트 대상 어드레스
- 3) 제 2 오퍼랜드(OP2): 쉬프트 할 비트 수  
상수의 범위 : 1 ~ 31
- 4) 결과출력 : W  
W = 0 : 정상인 경우에 W 는 0 이 됩니다.  
W = 1 : 에러인 경우에 W 는 1 이 됩니다.

[주 의]

SHR 는 산술 쉬프트로 OP1 의 최상위 비트(부호비트)를 우측으로 쉬프트 하더라도 최상위비트는 유지됩니다.

예 1] 1000\*\*\*00 → 1100\*\*\*00, 예 2] 1100\*\*\*00 → 1110\*\*\*00, 예 3] 0100\*\*\*00 → 0010\*\*\*00

## 6.27) SFL ( SUB 79: SHIFT LEFT )

[ 명령어: SUB 79 SFL ]

(기능)

Shift 대상 어드레스에 있는 데이터를 Shift 비트 오퍼랜드에 지정된 bit 만큼 좌측으로 SHIFT 합니다.

(형식)

| SFL | OP1             | OP2 |
|-----|-----------------|-----|
| OP1 | 쉬프트 대상 어드레스     |     |
| OP2 | 쉬프트 비트 데이터 어드레스 |     |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|----------------------------|---|---|---|---|---|---|---|-----|
|      | X                          | Y | R | G | F | D | T | C | INT |
| OP1  | X                          | O | O | O | X | O | X | O | X   |
| OP2  | X                          | O | O | O | X | O | X | O | O   |

(래더)

(설명)

제어조건

실행지령 : ACT

ACT = 0 : SFL 명령을 실행하지 않습니다.

ACT = 펄스 신호 : SFL 명령을 실행합니다.

⌞ 반드시 ACT=1 에서 명령 실행한 후, 바로 0 으로 해야만 합니다.

제 1 오퍼랜드(OP1) : 쉬프트 대상 어드레스

제 2 오퍼랜드(OP2) : 쉬프트할 비트수

상수의 범위 : 1 ~ 31

결과출력 : W

W = 0 : 정상인 경우에 W 는 0 이 됩니다.

W = 1 : 에러인 경우에 W 는 1 이 됩니다.

## 6.28) 시스템 매크로 변수 Read/Write 기능 명령어 설명

비저장형 매크로 변수 : #100 ~ #199

저장형 매크로 변수 : #200 ~ #699

PA 파라미터 맵핑 매크로 변수 : #1000 ~ #2999

PI 파라미터 맵핑 매크로 변수 : #3000 ~ #3999

SV map 맵핑 매크로 변수 : #4000 ~ #5999

SN map 맵핑 매크로 변수 : #6000 ~ #6999

PU 파라미터 맵핑 매크로 변수 : #9000 ~ #10999  
PM 파라미터 맵핑 매크로 변수 : #20000 ~ #29999

RDREG : 시스템 매크로 변수의 값을 원하는 개수 만큼 얻어와 (\*1000)을 하여 정수형으로 바꾼 후 OP3에 저장합니다.  
OP1 = 시스템 매크로 번호 (정수값 지령 혹은 어드레스 참조)  
OP2 = 매크로 개수 (0 ~ 31 개 까지만 처리)  
OP3 = 저장할 시작 어드레스  
WRREG : 오퍼랜드 1 의 값을 오퍼랜드 2 에 해당되는 시스템 매크로 변수의 값으로 설정합니다.  
OP1 = 정수값 혹은 어드레스 번호  
OP2 = 시스템 매크로 번호

적용되지 않는 매크로 번호  
G map 115 번지 0 ~ 31 bit : #7000 ~ 7031  
G map 115 번지 ~ 119 번지 : #7032 ~ 7035  
F map 105 번지 0 ~ 31 bit : #7500 ~ 7531  
F map 105 번지 ~ 109 번지 : #7532 ~ 7535

1) Turbohx1.FUN 의 추가된 내용. ( 2005. )

|    |   |   |   |   |   |       |     |                                        |
|----|---|---|---|---|---|-------|-----|----------------------------------------|
| 41 | 3 | 1 | 0 | 0 | 0 | RDREG | ANR | Read Macro Var. : Reg, cnt, Start Addr |
| 42 | 2 | 1 | 0 | 0 | 0 | WRREG | RN  | Write Macro Var. : Addr, Reg           |

2) Turbohx1.INI 에서 추가 된 내용

[Function41]

Input=ACT

Operand1=X,Y,R,G,F,D,T,C

Operand1Range=100,30000

Operand2=R,G,D,T,C

Operand2Range=0,32

Operand3=Y,R,G,D,T,C

[Function42]

Input=ACT

Operand1=X,Y,R,G,F,D,T,C

Operand1Range=-2147483648,2147483647

Operand2=Y,R,G,D,T,C

Operand2Range=100,30000

6.29) RDREG( SUB 41 : 시스템 매크로변수의 값 읽어오기 )

[명령어: SUB 41 RDREG]

(기능)

오퍼랜드 1 의 값에 해당되는 시스템 매크로 변수의 값을 오퍼랜드 2 의 개수 만큼 읽어온 후 각 값에 1000 을 곱한 후 정수형으로 바꾸어서 오퍼랜드 3 의 어드레스를 시작으로 입력합니다.

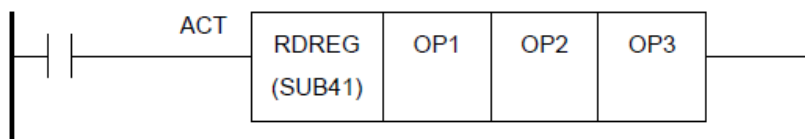
(형식)

|     | RDREG                            | OP1 | OP2 | OP3 |
|-----|----------------------------------|-----|-----|-----|
| OP1 | 시스템 매크로 번호 설정 오퍼랜드               |     |     |     |
| OP2 | 연속하여 읽어올 매크로 개수 (0 ~ 31 개까지만 처리) |     |     |     |
| OP3 | 출력 데이터 시작 어드레스                   |     |     |     |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능 X : 사용불가 ) |   |   |   |   |   |   |   | 정수 |
|------|----------------------------|---|---|---|---|---|---|---|----|
|      | X                          | Y | R | G | F | D | T | C |    |
| OP1  | X                          | O | O | O | X | O | X | O | X  |
| OP2  | X                          | O | O | O | X | O | X | O | O  |
| OP3  | X                          | O | O | O | X | O | X | O | X  |

(래더)



(설명)

제어조건

실행지령 : ACT

ACT=0 : RDREG 명령어를 실행하지 않습니다.

ACT=1 : RDREG 명령어를 실행합니다.

제 1 오퍼랜드(OP1) : 시스템 매크로 번호 설정 오퍼랜드

시스템 매크로 번호를 설정합니다. 직접 매크로 변수 번호를 정수로 입력하거나 또는 어드레스를 참조할 수 있습니다.

제 2 오퍼랜드(OP2) : 연속하여 읽어올 매크로 개수

제 1 오퍼랜드의 데이터를 시작으로 몇 개의 데이터를 연속해서 읽어올 것인지를 설정합니다. (주의) 이 값이 0 인 경우와 32 이상을 설정한 경우에는 기본적으로 오퍼랜드 1 에 해당되는 1 개의 데이터만을 읽어옵니다.

제 3 오퍼랜드(OP3) : 출력 데이터 시작 어드레스

제 1 오퍼랜드의 데이터를 저장하는 시작 어드레스입니다.

6.30) WRREG( SUB 42 : 시스템 매크로변수에 값 쓰기 )

[ 명령어: SUB 42 WRREG ]

(기능)

오퍼랜드 1 의 값을 오퍼랜드 2 에 해당되는 시스템 매크로 변수의 값으로 설정합니다.

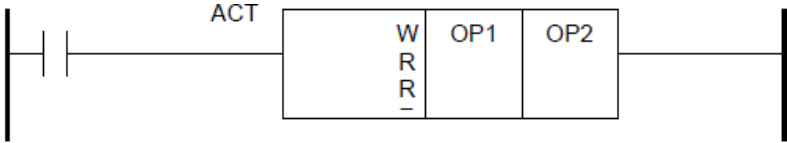
(형식)

| WRREG      OP1    OP2 |                      |
|-----------------------|----------------------|
| OP1                   | 입력 데이터 오퍼랜드          |
| OP2                   | 출력 데이터 시스템 매크로 변수 번호 |

(사용가능 어드레스)

| 오퍼랜드 | 어드레스 ( O : 사용가능    X : 사용불가 ) |   |   |   |   |   |   |   | 정수  |
|------|-------------------------------|---|---|---|---|---|---|---|-----|
|      | X                             | Y | R | G | F | D | T | C | INT |
| OP1  | O                             | O | O | O | O | O | X | O | O   |
| OP2  | X                             | X | X | X | X | X | X | X | O   |

(래더)



(설명)

제어조건

실행지령 : ACT

ACT=0 : WRREG 명령어를 실행하지 않습니다.

ACT=1 : WRREG 명령어를 실행합니다.

제 1 오퍼랜드(OP1): 입력 데이터 오퍼랜드

전송되어질 데이터를 지정합니다. (정수형, um 단위)

제 2 오퍼랜드(OP2): 출력 데이터 어드레스

제 1 오퍼랜드에 해당되는 값을 1000 으로 나누어 이 값을 실수형으로 바꾸어서 시스템 매크로 변수에 대입합니다.

[ Version V2.8 ]

200500901 – V2.8 (0901)-Kor/Eng

- 1) 영문 Windows 에서도 Ladder 편집을 할 수 있도록 편집기 메뉴의 영문화 진행함.
- 2) PLC function 정보와 관련된 파일의 경로를 변경함 (기존 PLCEditor.exe 아래 ‘data’ 폴더에 있는 \*.ini, \*.fun 파일들을 ‘ENG’ 폴더와 ‘KOR’ 폴더에 복사해 넣어야 합니다. ) – eng 폴더에는 영문 \*.fun 파일이 있어야 합니다.
- 3) HX 실행시에도 ‘PLC’ 폴더아래에 ‘ENG’ 폴더와 ‘KOR’ 폴더가 있어야 합니다.

4.4 HX PLC MESSAGE 파일 작성법

## 4.4.1 HX PLC MESSAGE

PLC 에서 검출하는 ALARM 및 WARNING 에 대해서는 G MAP 을 이용하여 CNC 로 알릴 수 있으며, 이 때 PLC 프로그래머는 G MAP 에 대응하는 ALARM 또는 WARNING MESSAGE DATA FILE 를 작성하여 800S 시스템 폴더 안에 PLC 폴더에서 보관해야 합니다.

ALARM 또는 WARNING 이 발생되면 화면에 디스플레이가 되어 사용자에게 알릴 수 있게 됩니다. 특히 OVER TRAVEL 등의 안전과 관계되는 ALARM 에 대해서는 CNC 에 비상정지(EMERGENCY) 신호를 입력하는 등의 조치가 반드시 필요합니다.

| 분 류     | G MAP 영역          | MESSAGE DATA FILE |
|---------|-------------------|-------------------|
| ALARM   | G900.00 ~ G949.1F | PLCAImDt.txt      |
| WARNING | G950.00 ~ G999.1F | PLCOpDt.txt       |

## 4.4.2 ALARM MESSAGE (PLCAImDt.txt)

ALARM MESSAGE 는 PLC 를 작성할 때 G900 ~ G949 Address 에 할당해서 작성합니다.

ALARM MESSAGE 는 비상정지(EMERGENCY-STOP)나 OVER TRAVEL 등 기계 작동에 문제가 발생하는 경우 그 원인을 작업자에게 알려주기 위해 작성합니다.

ALARM MESSAGE 는 알람 리스트에 기록을 남길 수 있습니다.

ALARM MESSAGE 는 PLCAImDt.txt 파일에 화면에 디스플레이 되는 메시지 내용을 작성합니다.

ALARM MESSAGE 는 빨간 바탕에 하얀 글씨로 화면 상단에서 깜박입니다.

### [작성 예]

| 작성 내용                      | 설 명                                                                                                   |
|----------------------------|-------------------------------------------------------------------------------------------------------|
| # count                    | ‘#’ 은 주석문을 의미합니다.                                                                                     |
| 2                          | 알람 메시지의 갯수를 정확히 기입합니다.                                                                                |
| # code bit msg             | 어드레스 + 점점비트(0~31)+알람 내용기억 유/무(1/0)                                                                    |
| 900 0 0 비상 정지 상태           | 예 1] 900(공백) 0(공백) 0(공백) 비상 정지 상태<br>“G900.00 을 HIGH 로 하면 화면에 알람이 디스플레이되나 PLC 알람 내용은 기억되지 않습니다.       |
| 900 1 1 X AXIS OVER TRAVEL | 예 2] 900(공백) 1(공백) 1(공백) X AXIS OVER TRAVEL<br>“G900.01 을 HIGH 로 하면 화면에 알람이 디스플레이되며 PLC 알람 내용은 기억됩니다. |
| -1                         | ‘-1’을 사용하여 파일의 끝을 기입합니다.                                                                              |

### 예제)

```
# count
5
# code bit msg
900 0 0 비상정지 중 입니다.
900 9 1 X 축 (Loading-r) + O.T 중 입니다.
900 10 1 Y 축 (Loading-r) - O.T 중 입니다.
900 11 1 Z 축 (Unloading-r) + O.T 중 입니다.
900 29 1 공기 압력이 부족합니다.
-1
```

## 4.4.3 WARNING MESSAGE (PLCOpDt.txt)

WARNING MESSAGE 는 PLC 를 작성할 때 G950 ~ G999 Address 에 할당해서 작성합니다.  
WARNING MESSAGE 는 원점 복귀 중이나 원점복귀 미완료 등 기계 작동에는 문제가 없으나 잘못 조작 하였거나 현재 상태를 작업자에게 알려주기 위해 작성합니다.  
WARNING MESSAGE 는 알람 리스트에 기록을 남길 수 없습니다.  
WARNING MESSAGE 는 PLCOpDt.txt 파일에 화면에 디스플레이 되는 메시지 내용을 작성합니다.  
WARNING MESSAGE 는 노란 바탕에 까만 글씨로 화면 하단에서 깜박입니다.

[작성 예]

| 작성 내용                    | 설 명                                                                             |
|--------------------------|---------------------------------------------------------------------------------|
| # count                  | ‘#’ 은 주석문을 의미합니다.                                                               |
| 2                        | 경고 메시지의 갯수를 정확히 기입합니다.                                                          |
| # code   bit msg         | 어드레스 + 점점비트(0~31) + 알람 내용                                                       |
| 950 00 원점복귀 중 입니다.       | 예 1] 950(공백) 0(공백) 원점복귀 중 입니다.<br>“G950.00”을 HIGH 로 하면 화면에 경고가 디스플레이 됩니다        |
| 950 01 원점복귀가 완료되지 않았습니다. | 예 2] 950(공백) 1(공백) 원점복귀가 완료되지 않았습니다.<br>“G950.01”을 HIGH 로 하면 화면에 알람이 디스플레이 됩니다. |
| -1                       | ‘-1’을 사용하여 파일의 끝을 기입합니다.                                                        |

예제)

```
# count
  2
# code   bit msg
950  0   원점복귀가 완료되지 않았습니다.
950  1   원점복귀 중 입니다.
-1
```